GRMS Website for FUUAST

Developed By

Alishba Arshad (B.S CS Aut-2022-M-B-0074)

Sana Tariq (B.S CS Aut-2022-M-B-0111)

BS (Computer Science)

Supervised By

Dr. Kashif Rizwan

Department of Computer Science

Federal Urdu University of Arts, Science & Technology

Islamabad

SESSION [2022-2026]

GRMS Website For FUUAST

A project submitted to the Department of Computer Science as partial fulfillment of the requirement for the award of Degree of Bachelor of Science in Computer Science.

Name	Registration Number
Alishba Arshad	**37333**
Sana Tariq	**37963**

Supervisor

Dr. Kashif Rizwan

Lecturer

Department of Computer Science

Federal Urdu University of Arts, Science and Technology

Islamabad

Final Approval

It is certified that I have read the project report submitted by **Alishba Arshad** (37333) and **Sana Tariq** (37963) and it is our judgment that this project is of sufficient standard to warrant its acceptance by the FUUAST university, Islamabad for the Degree of Bachelor of Science in Computer Science.

Committee:

External Examiner _____

Project Coordinator _____

Mr. Khawaja Tahir Mahmood

Lecture

Department of Computer Science

Federal Urdu University of Arts, Science

and Technology

Islamabad

Supervisor _____

Dr. Kashif Rizwan

Lecturer

Department of Computer Science

Federal Urdu University of Arts, Science

and Technology Islamabad

Declaration

We hereby declare that this software, neither as a whole nor as a part has been copied out from any source. It is further declared that we have developed this software and the accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted in support of any application for any other degree or qualification of this or any other university or institute of learning.

Alishba Arshad

Sana Tariq

Dedicated to our beloved

parents,

Teachers

And

The fellows

Acknowledgement

Thanks to Almighty Allah for giving us knowledge, power and strength to accomplish this task. We learned a lot while doing this project and this will certainly help us in our forthcoming life. Many of our teachers helped us during this project but we are really thankful to our supervisor Dr. Kashif Rizwan for his help and support in all phases of our project. His encouragement helped us a lot during times of difficulties. In the end we would like to say thanks to our parents for their support and encouragement.

Alishba Arshad

Sana Tariq

Project in Brief

Project Title	GRMS Website For FUUAST
Organization	Federal Urdu University of Arts, Science and Technology
Objectives	• Secure Cloud Delivery • Precise Excess Matrix • Automated Document Engine • Fast Collaboration
Undertaken By	Alishba Arshad Sana Tariq
Supervised By	Dr. Kashif Rizwan
Date Started	Oct, 15, 2025
Date Completed	June ,25, 2026
Technologies Used	• Django (Python) • HTML5 • CSS3 • JavaScript • Bootstrap 5
System used	11th Gen Core i5 Processor @ 2.6 GHz, 8 GB RAM, 256 SSD

ABSTRACT

The Graduate Research Management System (GRMS) is a comprehensive web-based platform developed to digitalize and streamline the entire research lifecycle at Federal Urdu University of Arts, Science and Technology (FUUAST). The system is designed to automate the manual research workflows including student admissions, synopsis submission, thesis evaluation, progress tracking, and degree issuance, ensuring efficiency, transparency, and data integrity across all academic research activities.

The system supports four distinct user roles: Admin, Student, Supervisor, and Examiner, each with customized dashboards and role-based access control. Students can submit synopses, progress reports, theses, request extensions, schedule meetings, and apply for degree letters. Supervisors can review synopses, provide feedback on progress reports, evaluate theses, and manage student meetings. Examiners are assigned to theses for independent evaluation, where they can submit marks, recommendations, and detailed reports. The admin has complete control over user management, product oversight, and system monitoring.

The platform includes a degree letter generation module, which allows admins to verify student eligibility and generate official degree letters in PDF format, enabling automated degree issuance.

The system uses MySQL for secure data storage, Django for backend development, and Bootstrap for a responsive frontend interface. It supports file uploads with validation for PDF documents and maintains version history for synopsis and thesis submissions. The system is deployed on PythonAnywhere, ensuring 24/7 accessibility for all users.

Overall, GRMS provides a complete digital ecosystem for managing graduate research, reducing manual paperwork, minimizing errors, improving communication, and ensuring timely completion of academic milestones. It serves as a scalable solution that can be adapted for multi-department and multi-campus environments, making it a valuable tool for modern academic institutions.

TABLE OF CONTENTS

Chapter # 1

Introduction

Revision History

Version	Description	Author	Date
1.0	This document contains Project Proposal of GRMS for FUUAST	Alishba Arshad & Sana Tariq	20th October 2025

GRMS Website For FUUAST

1.1 Introduction

Graduate Research Management System (GRMS) is a web-based application designed to manage, automate, and monitor all the research-related academic activities of postgraduate and undergraduate students at Federal Urdu University.

Graduate Research Management Committee (GRMC) is currently, managing research activities such as student admission, synopsis submissions, thesis evaluation, and viva results involves multiple manual steps; consuming time and causing miscommunication between students, supervisors, and administration.

This system aims to centralize all operations related to research management under one digital platform. It ensures smooth workflow, accurate data tracking, and transparency throughout the research lifecycle from admission to final degree issuance.

1.2 Problem Statement

At FUUAST, research-related processes like admission, synopsis submission, thesis evaluation, and viva communication are mostly manual or handled through basic spreadsheets and paper files. This results in several challenges.

Difficulty in maintaining research student records.

- Delays in communication between departments and supervisors.
- Lack of centralized data for tracking research progress.
- Manual approval and verification process lead to inconsistencies.
- No system for tracking extension cases or generating timely notifications.

Therefore, there is a need for an automated web-based Graduate Research Management System that can efficiently manage and monitor all research-related operations in a secure, centralized manner.

1.3 Proposed System

The proposed GRMS for FUUAST will be a web-based centralized management system that enables the university administration, students, and supervisors to manage research-related workflows seamlessly.

The system will allow users to perform digital record management for admissions, synopsis, thesis evaluation, and viva reports, along with real-time notifications and security management.

a) Key Features

- Automated student and user management.
- Secure authentication and password management.
- Real-time notifications for tasks and approvals.
- Digital storage of thesis, seminar reports, and viva results.
- Meeting record and extension case management.
- Easy report generation and document tracking.

b) Benefits of proposed system

- Saves time and reduces manual workload.
- Ensures accurate and consistent record keeping.
- Provides secure centralized data storage.
- Improves communication among students, supervisors, and admin.
- Enhances transparency in research progress tracking.
- Allows anytime, anywhere web access.
- Reduces paper usage (eco-friendly).
- Auto-notifications for deadlines and updates.
- Easy monitoring and reporting of student progress.
- Scalable and adaptable for future upgrades.

1.4 Scope

The GRMS is specifically designed for the Federal Urdu University research department to manage postgraduate and undergraduate research operations. However , its modular and scalable design allows expansion to multiple campuses or integration with other academic management systems in the future.

The system supports:

Web access for students, supervisors, and admin users.

- Secure document upload, review, and archiving.
- Automatic notification of deadlines, extensions, and results.
- Meeting management and case extension tracking.

1.5 Survey Analysis

Module / Feature	Oxford	Manchester	MIT	LUMS	QAU	COMSATS	GRMS (Proposed)
a. Admission / Enrollment Management	—	✓ [2]	✓ [3]	✓ [4]	✓ [5]	✓ [6]	✓
b.Student Management	—	—	—	—	—	—	✓
c. Security Management (login/password)	—	—	—	✓ [4]	✓ [5]	✓ [6]	✓
d. Supervisor Record Management	✓ [1]	✓ [2]	✓ [3]	✓ [4]	—	—	✓
e. Synopsis Management	—	—	—	✓ [4]	—	—	✓
f. Progress Report Management	✓ [1]	✓ [2]	—	✓ [4]	—	—	✓
g. Thesis Evaluation / Submission	—	—	✓ [3]	✓ [4]	—	✓ [6]	✓
h. Meeting Records Management	—	✓ [2]	—	—	—	—	✓
i. Email Notification Management	✓ [1]	✓ [2]	—	✓ [4]	—	—	✓

j. Extension Case Management	✓ **[1]**	✓ **[2]**	—	✓ **[4]**	—	—	✓
k. Degree Letter Management	—	—	—	—	—	—	✓

Legend:

✓ = **Evidenced on an official university page**

— = **No verifiable public evidence found on cited pages.**

1.6 Modules & Sub-Modules

a) Admission / Enrollment Management

- Add Student
- Search Student
- Update Student
- View All Students

b) Student Management

- Add Student
- Search Student
- Update Student
- View All Student
- Assign Supervisor

c) Security Management

- Process Login
- Change Password
- Reset Password

d) Supervisor Record Management

- Add / Update Supervisor Details

- View Supervised Student

e) Synopsis Management

- Add Synopsis
- Search Synopsis
- Update Synopsis
- View All Synopsis
- Pending Synopsis List

f) Progress Report Management

- Submit Progress Report
- View All Reports

g) Thesis Evaluation Management

- Submit Thesis
- Attach Evaluation Documents
- Upload Viva Results
- Handle Objection Letters & Replies

h) Meeting Records Management

- Add Meeting
- Record Discussion Points
- Upload Minutes of Meeting

i) Email Notification Management

- Auto-notifications for tasks and deadlines
- View All Notifications

j) Extension Case Management

- Add Case
- Update Case

- View All Cases
- Approve / Reject Case
- Auto-Check End Degree Timeframe

k) Degree Letter Management

- Generate Degree Letter
- Verify Completion

1.7 Actors

1. Primary Actors

Admin

Student

Supervisor

Department Coordinator

2. Secondary Actors

Evaluation Committee

External Examiner

Registrar / Examination Branch

System Administrator

1.8 Tools & Technologies

Here are the tools and technologies that we may use in the development of this project:

Frontend	HTML5, CSS3, JavaScript, Bootstrap 5
Backend	Django (Python Framework)
Database	MySQL
IDE	Visual Studio Code
Version Control	GitHub
Web Server	WAMP Server

OS	Windows
Scheduling	MS Project

1.9 System Design Approach & Process Model Used

The system design follows the Modular and Layered Architecture, ensuring each feature can be independently developed and maintained.

The architecture includes:

Presentation Layer: User interfaces for admin, student, and supervisor.

Application Layer: Business logic implemented in Django views and models.

Database Layer: Structured storage of all entities like students, synopsis, thesis, etc.

Security Layer: Handles authentication, authorization, and encryption

The Iterative Development Model will be used for this project**.**

1.10 Limitation/Constraint

- Requires stable internet connection for users.
- Initial data migration from paper to system may be time-consuming.
- Limited to web access; mobile application version not included.
- Notifications dependent on email/SMS gateway services

Chapter # 2

Analysis

SOFTWARE REQUIREMENT SPECIFICATION

FOR

GRMS FOR FUUAST

Version 1.1

Prepared By

Alishba Arshad

Sana Tariq

27 /11/2025

REVISION HISTORY

Version	Description	Author	Date
1.0	This covers the major SRS document	Alishba Arshad & Sana Tariq	27/11/2025
1.1	This covers the major SRS document	Alishba Arshad & Sana Tariq	27/11/2025

2.1 SOFTWARE REQUIREMENTS SPECIFICATIONS (SRS):

2.1.1 Introduction

Graduate Research Management System (GRMS) is a web-based application designed to manage, automate, and monitor all the research-related academic activities of postgraduate and undergraduate students at Federal Urdu University.

Graduate Research Management Committee (GRMC) is currently, managing research activities such as student admission, synopsis submissions, thesis evaluation, and viva results involves multiple manual steps; consuming time and causing miscommunication between students, supervisors, and administration.

This system aims to centralize all operations related to research management under one digital platform. It ensures smooth workflow, accurate data tracking, and transparency throughout the research lifecycle from admission to final degree issuance.

2.1.2 Purpose

The purpose of the Graduate Research Management System (GRMS) for FUUAST is to create a centralized, web-based platform that digitizes and streamlines all research-related activities across the university. The system is designed to handle workflows such as admissions, synopsis submission, thesis evaluation, viva results, and document tracking in a fully digital environment.

It also aims to improve security through authenticated access and reduce the workload caused by manual paperwork. By providing a centralized repository for research documents, automated user management, and transparent progress tracking, GRMS helps ensure accuracy, efficiency, and easy access for students, supervisors, and administrative staff. Ultimately, it supports a faster, more organized, and eco-friendly research management process that can scale according to future university needs.

This platform will:

- Centralized and web-based platform for all research workflows.
- Digital handling of admissions, synopsis, thesis evaluation, and viva reports.
- Secure authentication and password management.
- Automated user and student record management.

- Digital storage for thesis, seminar reports, and viva outcomes.
- Meeting record management and extension case tracking.
- Reduces manual workload and paper usage.
- Improves accuracy, transparency, and communication.
- Allows anytime, anywhere access through a web interface.
- Scalable for future upgrades and university needs.

2.1.3 Scope

The scope of the Graduate Research Management System (GRMS) covers the complete digital management of postgraduate and undergraduate research activities within the Federal Urdu University research department.

The system provides web-based access for students, supervisors, and administrative users to manage research workflows such as document submission, review, approvals, meeting records, notifications, and progress tracking.

It enables secure uploading and archiving of research documents while ensuring automated communication for deadlines, extensions, and evaluation results. Although designed specifically for FUUAST, its modular and scalable architecture allows future expansion to multiple campuses and integration with other academic or departmental systems.

The functional scope of the GRMS system includes the following main features:

- Admission Management
- Student Management System
- User Security Management
- Synopsis Submission Management
- Progress Report Management
- Thesis Submission Management
- Meeting Record Management
- Extension Case Management

- Degree Letter

2.1.4 Definitions, Acronyms, and Abbreviations

Abbreviation	Definition
GRMS	Graduate Research Management System
FUUAST	Federal Urdu University Arts , Science and Technology

2.1.5 Overview

The Graduate Research Management System (GRMS) is a web-based platform designed to streamline and automate all research-related processes at FUUAST. It enables students, supervisors, and administrators to manage admissions, synopsis submissions, thesis evaluations, meetings, notifications, and research documents in a centralized and secure environment.

By replacing manual paperwork with digital workflows, GRMS improves efficiency, enhances communication, ensures transparency, and provides a structured way to track research progress from admission to degree completion.

2.2 Functional Requirements

2.2.1 Admission Management

2.2.1.1 Process Add New Student

SRS-1	The system shall allow admin users to register a new student by providing required information (e.g., full name, student ID, email, contact number, program, department, session).
SRS-2	The system shall validate that all mandatory fields are filled correctly before creating a new student record.

SRS-3	The system shall ensure that student ID and email are unique for each student account.
SRS-4	Upon successful registration, the system shall securely store student information in the database.
SRS-5	The system shall send a confirmation notification to the admin after student registration is completed.

2.2.1.2 Process Update Student Information

SRS-6	The system shall allow admin users to edit or update student information including contact details, program, session, and supervisor.
SRS-7	The system shall validate updated information before saving changes.
SRS-8	The system shall maintain a history log of important updates (e.g., supervisor changes, program changes).

2.2.1.3 Process Search Student Records

SRS-9	The system shall allow admin users to assign or update the supervisor for a selected student.

SRS-10	The system shall notify the assigned supervisor and student automatically via email/notification.

2.2.1.4 Process Assign Supervisor

SRS-11	The system shall allow admin users to assign or update the supervisor for a selected student.
SRS-12	The system shall notify the assigned supervisor and student automatically via email/notification.

2.2.1.5 Process View Enrollment Status

SRS-13	The system shall display the enrollment status of each student (Active, Inactive, Completed, Withdrawn).
SRS-14	The system shall allow authorized users to update a student's status when required.

2.2.1.6 Process Export Student Data

SRS-15	The system shall allow admin users to export student records in PDF or Excel format.

2.2.2 Student Management

2.2.2.1Process View Student Profile

SRS-16	The system shall allow authorized users to view complete student profiles, including personal, academic, and research-related information.
SRS-17	The system shall retrieve and display student data from the database in real time.
SRS-18	The system shall display profile sections such as personal information, academic records, synopsis status, thesis progress, and supervisor details.

2.2.2.2 Process Update Student Profile

SRS-19	The system shall allow admin users to update selected fields in a student's profile (e.g., contact info, department, session, supervisor).
SRS-20	The system shall restrict sensitive updates (e.g., program level, department) to admin-only permissions.
SRS-21	The system shall validate updated information before saving the changes.

2.2.2.3Process Manage Academic Records

SRS-22	The system shall allow admin users to add or update academic records such as coursework, semester GPA, or research progress milestones.

SRS-23	The system shall store academic records securely and link them to the student's main profile.
SRS-24	The system shall allow authorized users to view academic history through a structured interface.

2.2.2.4 Process Manage Research Progress

SRS-25	The system shall allow supervisors to update student research progress, including synopsis approval status, report submissions, and thesis stage.
SRS-26	The system shall allow students to view progress updates in their dashboard.
SRS-27	The system shall generate alerts for pending tasks or delays in research progress.

2.2.2.5 Process Student Status Management

SRS-28	The system shall allow admin users to update student status (Active, Inactive, Completed, Withdrawn).
SRS-29	The system shall display the student's current status on their profile and dashboard.

SRS-30	The system shall record every status change in the activity logs.

2.2.2.6 Process Search & Filter Students

SRS-31	The system shall allow users to search students using multiple filters such as name, student ID, program, department, supervisor, or status.
SRS-32	The system shall display search results in a paginated table for easy browsing.
SRS-33	The system shall allow exporting filtered search results in PDF or Excel format.

2.2.3 Login & Security Management

2.2.3.1Process User Sign Up

SRS-34	The system shall allow new users (students, supervisors, admin) to create an account by providing required details such as name, email, phone number, and role.
SRS-35	The system shall validate that all required fields are filled correctly before account creation.
SRS-36	The system shall ensure that email and phone number are unique for each user account.

SRS-37	Upon successful registration, the system shall securely store user data and redirect them to the login page.
SRS-38	The system shall send a confirmation email or message upon successful registration.

2.2.3.2Process User Login

SRS-39	The system shall allow users to log in using their registered email and password.
SRS-40	The system shall authenticate user credentials before granting system access.
SRS-41	The system shall display an error message if the email or password is incorrect.
SRS-42	The system shall restrict login attempts after multiple failed tries for security.

2.2.3.3 Process Password Management

SRS-43	The system shall allow users to reset their password by using the “Forgot Password” option.

SRS-44	The system shall send a password reset link to the registered email, valid for a limited time.
SRS-45	The system shall enforce password rules (minimum 8 characters, one uppercase letter, one number, one special character).

2.2.3.4 Process Role-Based Access Control

SRS-46	The system shall assign roles to users such as Student, Supervisor, Co-Supervisor, Admin, and GRMC staff.
SRS-47	The system shall ensure that each role has access only to authorized modules and actions.
SRS-48	The system shall prevent unauthorized users from viewing or editing restricted information.

2.2.3.5 Process Account Security

SRS-49	The system shall use encryption to store passwords securely in the database.
SRS-50	The system shall automatically log out inactive users after a predefined time.

SRS-51	The system shall maintain an activity log for login attempts, password changes, and security events.

2.2.3.6 Process User Account Verification

SRS-52	The system shall require email verification before allowing a new user to fully access the system.
SRS-53	The system shall display a notification if the user attempts to log in without verifying their email.
SRS-54	The system shall lock accounts that appear suspicious or show unusual login activity.

2.2.4 Supervisor Record Management

2.2.4.1 Process Add Supervisor Record

SRS-55	The system shall allow admin users to add a new supervisor record by entering details such as name, designation, department, email, and contact number.
SRS-56	The system shall validate that all required supervisor information is provided before saving the record.
SRS-57	The system shall ensure that the supervisor's email and employee ID (if applicable) are unique.

2.2.4.2 Process Update Supervisor Information

SRS-58	The system shall allow admin users to update supervisor details such as contact information, department, and designation.
SRS-59	The system shall validate updated information before saving changes.
SRS-60	The system shall maintain a log of changes made to supervisor records.

2.2.4.3 Process Search Supervisor Records

SRS-61	The system shall allow users to search supervisors using filters such as name, department, designation, or email.
SRS-62	The system shall display search results in a paginated and sortable list.
SRS-63	The system shall allow exporting supervisor search results in PDF or Excel format.

2.2.4.4 Process Assign Supervisors to Students

SRS-64	The system shall allow admin users to assign a supervisor to a student from the list of active supervisors.

SRS-65	The system shall ensure that a supervisor cannot exceed the maximum allowed number of supervisees (if defined by department rules).
SRS-66	The system shall notify both the supervisor and student upon successful assignment.

2.2.4.5 Process Manage Supervisor Availability

SRS-67	The system shall allow supervisors to set their availability status (Available / Not Available / On Leave).
SRS-68	The system shall display supervisor availability during student–supervisor assignment.
SRS-69	The system shall restrict assignment of unavailable supervisors.

2.2.4.6 Process Reporting & Record Management

SRS-70	The system shall generate reports on supervisor workload (number of students assigned, department-wise distribution).
SRS-71	The system shall allow admin users to export supervisor records and workload reports in PDF or Excel format.

SRS-72	The system shall maintain a digital archive of supervisor records for administrative use.

2.2.5 Synopsis Management

2.2.5.1 Process Submit Synopsis

SRS-73	The system shall allow students to submit their synopsis online by uploading a PDF file along with required details (title, supervisor, submission date).
SRS-74	The system shall validate that all mandatory information is provided before allowing submission.
SRS-75	The system shall ensure that only PDF format is accepted for synopsis submission.
SRS-76	Upon successful submission, the system shall store the synopsis securely in the database.
SRS-77	The system shall send an automatic notification to the supervisor and admin after the student submits the synopsis.

2.2.5.2 Process Review Synopsis

SRS-78	The system shall allow supervisors and admin users to review the submitted synopsis.
SRS-79	The system shall allow reviewers to add comments, feedback, or required changes to the synopsis.
SRS-80	The system shall allow reviewers to change the synopsis status to *Pending*, *Approved*, or *Rejected*.

2.2.5.3 Process Upload Revised Synopsis

SRS-81	The system shall allow students to upload a revised version of the synopsis if the previous version was rejected or required changes.
SRS-82	The system shall maintain a version history of all submitted synopsis files.
SRS-83	The system shall notify the supervisor when a revised synopsis is uploaded.

2.2.5.4 Process Synopsis Approval Workflow

SRS-84	The system shall route the synopsis to the appropriate supervisor and admin for approval.

SRS-85	The system shall track the progress of the synopsis review workflow and display the current status to the student.
SRS-86	The system shall generate alerts for pending approvals and overdue review actions.

2.2.5.5 Process View Synopsis Status

SRS-87	The system shall allow students to view the status of their synopsis (Submitted, Under Review, Approved, Rejected).
SRS-88	The system shall display supervisor comments and instructions for revision (if any).
SRS-89	The system shall allow admin users to view and filter synopsis records based on status, department, supervisor, or student.

2.2.5.6 Process Synopsis Reporting & Record Management

SRS-90	The system shall generate reports for submitted, approved, and rejected synopses.
SRS-91	The system shall allow admin users to export synopsis reports in PDF or Excel format.
SRS-92	The system shall maintain digital archives of all synopsis records for future reference.

2.2.6 Progress Report Management

2.2.6.1 Process Submit Progress Report

SRS-93	The system shall allow students to upload their progress report in PDF format along with required details such as semester, submission date, and description of work completed.
SRS-94	The system shall validate all mandatory fields before allowing the student to submit the progress report.
SRS-95	The system shall ensure that only PDF files are accepted for progress report submissions.
SRS-96	Upon successful submission, the system shall securely store the report in the database.
SRS-97	The system shall send an automatic notification to the assigned supervisor after the student submits the progress report.

2.2.6.2 Process Review Progress Report

SRS-98	The system shall allow supervisors to review progress reports submitted by their assigned students.

SRS-99	The system shall allow supervisors to add comments, feedback, or required improvements.
SRS-100	The system shall allow supervisors to set the report status as *Accepted*, *Needs Improvement*, or *Rejected*.

2.2.6.3 Process Upload Revised Report

SRS-101	The system shall allow students to upload a revised progress report if the supervisor requests changes.
SRS-102	The system shall maintain version history for each progress report submitted or revised.
SRS-103	The system shall notify the supervisor when the student uploads a revised report.

2.2.6.4 Process Track Report Status

SRS-104	The system shall allow students to view the status of their progress report (Submitted, Under Review, Accepted, Needs Improvement, Rejected).
SRS-105	The system shall display supervisor comments and improvement instructions.

SRS-106	The system shall allow admin users to view and filter progress report records by department, student, semester, supervisor, or status.

2.2.6.5 Process Progress Evaluation Workflow

SRS-107	The system shall track the complete workflow of progress report submission and evaluation.
SRS-108	The system shall send reminders to supervisors for pending or overdue report evaluations.
SRS-109	The system shall automatically update the student's research progress dashboard after evaluation.

2.2.6.6 Process Reporting & Record Management

SRS-110	The system shall generate departmental or program-wise progress report summaries.
SRS-111	The system shall allow admin users to export progress report data in PDF or Excel format.
SRS-112	The system shall maintain a digital archive of all progress reports for future audit or reference purposes.

2.2.7 Thesis Evaluation Management

2.2.7.1 Process Submit Thesis

SRS-113	The system shall allow students to upload their thesis document in PDF format for evaluation.
SRS-114	The system shall validate required fields such as thesis title, supervisor name, submission date, and student ID before allowing submission.
SRS-115	The system shall ensure that only PDF format is accepted for thesis submissions.
SRS-116	The system shall securely store the thesis file in the database upon successful submission.
SRS-117	The system shall notify the supervisor and admin about the thesis submission.

2.2.7.2 Process Assign Thesis Examiners

SRS-118	The system shall allow admin users to assign internal and external examiners to evaluate the student's thesis.
SRS-119	The system shall allow admin users to modify assigned examiners when necessary.

SRS-120	The system shall notify all assigned examiners via email/notification after assignment.

2.2.7.3 Process Upload Examiner Reports

SRS-121	The system shall allow examiners to upload their evaluation reports, comments, and scores in PDF format.
SRS-122	The system shall validate the file format before accepting the evaluation report.
SRS-123	The system shall maintain version history for any updated or revised evaluation reports.

2.2.7.4 Process Thesis Evaluation Workflow

SRS-124	The system shall track the complete workflow of thesis evaluation including submission, review, and acceptance.
SRS-125	The system shall allow supervisors and admin to mark the thesis status as Under Review, Minor Changes Required, Major Changes Required, or Accepted.
SRS-126	The system shall notify the student about the evaluation decision and required changes.

2.2.7.5 Process Upload Revised Thesis

SRS-127	The system shall allow students to upload a revised thesis after receiving feedback from examiners.
SRS-128	The system shall maintain version control by storing all previous versions of the thesis.
SRS-129	The system shall notify the assigned examiners upon the upload of a revised thesis.

2.2.7.6 Process Generate Thesis Evaluation Summary

SRS-130	The system shall generate a summary report including examiner comments, scores, decisions, and thesis version history.
SRS-131	The system shall allow admin users to export the thesis evaluation summary in PDF or Excel format.
SRS-132	The system shall store final evaluation reports and thesis documents in a digital archive for future reference.

2.2.8 Meeting Record Management

2.2.8.1 Process Schedule Meeting

SRS-133	The system shall allow supervisors to schedule a meeting with a student by selecting date, time, mode (online/physical), and meeting agenda.
SRS-134	The system shall validate required fields before scheduling a meeting.
SRS-135	The system shall notify the student and supervisor through the system and email when a meeting is scheduled.

2.2.8.2 Process Record Meeting Details

SRS-136	The system shall allow supervisors to record meeting details including summary, tasks assigned, and progress discussed.
SRS-137	The system shall allow supervisors to upload related files or documents for meeting evidence.
SRS-138	The system shall store meeting records securely in the database with timestamps.

2.2.8.3 Process Update Meeting Record

SRS-139	The system shall allow supervisors to update or edit meeting records if corrections or additions are required.

SRS-140	The system shall maintain a version history for updated meeting records.

2.2.8.4 Process View Meeting History

SRS-141	The system shall allow students and supervisors to view complete meeting history including summaries, feedback, and assigned tasks.
SRS-142	The system shall allow admin users to view meeting records for any student when needed.
SRS-143	The system shall allow filtering of meeting records by date, supervisor, topic, or student.

2.2.8.5 Process Meeting Reminders

SRS-144	The system shall send automated reminders to students and supervisors before the scheduled meeting time.
SRS-145	The system shall send alerts for missed meetings or pending meetings requiring updates.

2.2.8.6 Process Reporting & Record Management

SRS-146	The system shall generate reports on meeting frequency, meeting outcomes, and supervisor–student engagement.

SRS-147	The system shall allow admin users to export meeting reports in PDF or Excel format.
SRS-148	The system shall archive all meeting records digitally for future reference and official use.

2.2.9 Extension Case Management

2.2.9.1 Process Submit Extension Request

SRS-149	The system shall allow students to submit an extension request by providing required details such as reason for extension, supporting documents, and requested extension duration.
SRS-150	The system shall validate all mandatory fields before allowing submission of the extension request.
SRS-151	The system shall allow students to upload supporting files in PDF format only.
SRS-152	The system shall notify the supervisor and admin when a student submits an extension request.

2.2.9.2 Process Review Extension Request

SRS-153	The system shall allow supervisors and admin to review submitted extension requests.
SRS-154	The system shall allow reviewers to add comments or request additional information.
SRS-155	The system shall allow reviewers to approve or reject the extension request.
SRS-156	The system shall update the request status to Pending, Approved, or Rejected accordingly.

2.2.9.3 Process Upload Revised Documents

SRS-157	The system shall allow students to upload revised documents if the supervisor or admin requests changes.
SRS-158	The system shall maintain version history for all uploaded extension-related documents.
SRS-159	The system shall notify the reviewer when revised documents are uploaded.

2.2.9.4 Process Update Extension Decision

SRS-160	The system shall allow admin users to finalize and record the decision regarding the extension request.

SRS-161	The system shall update the student's research timeline automatically based on the approved extension duration.
SRS-162	The system shall allow admin users to overturn or modify extension decisions when necessary.

2.2.9.5 Process View Extension Request Status

SRS-163	The system shall allow students to view the status of their extension requests (Submitted, Under Review, Approved, Rejected).
SRS-164	The system shall display reviewer comments and instructions for revision (if applicable).
SRS-165	The system shall allow admin users to filter extension requests by department, student, supervisor, or request status.

2.2.9.6 Process Reporting & Record Management

SRS-166	The system shall generate extension case reports based on program, department, supervisor, or timeframe.
SRS-167	The system shall allow admin users to export extension case reports in PDF or Excel format.

SRS-168	The system shall archive all extension case documents and decisions for future reference or audits .

2.2.10 Degree Letter File Management

2.2.10.1 Process Submit Degree Issuance Request

SRS-169	The system shall allow students to submit a request for degree issuance after completing all research requirements.
SRS-170	The system shall validate that the student has completed all mandatory steps (thesis approval, viva results, clearance) before allowing submission.
SRS-171	The system shall allow students to upload required supporting documents in PDF format.
SRS-172	The system shall notify admin and the concerned department when a degree issuance request is submitted.

2.2.10.2 Process Verify Degree Requirements

SRS-173	The system shall allow admin users to verify the student's academic and research records before processing the degree issuance request.

SRS-174	The system shall allow supervisors and admin to confirm completion of viva, thesis approval, and plagiarism clearance.
SRS-175	The system shall update the verification status as Pending, Verified, or Rejected.

2.2.10.3 Process Generate Degree Letter File

SRS-176	The system shall allow admin users to generate a degree issuance letter file containing student details, program details, approval dates, and verification information.
SRS-177	The system shall allow customization of degree letter templates according to university standards.
SRS-178	The system shall automatically insert verified information into the letter file.

2.2.10.4 Process Approve or Reject Degree Letter

SRS-179	The system shall allow authorized admin or registrar staff to approve or reject the generated degree letter.
SRS-180	The system shall allow reviewers to add comments or corrections before final approval.

SRS-181	The system shall notify the student once the degree letter is approved or rejected.

2.2.10.5 Process Download and Archive Degree Letter

SRS-182	The system shall allow students and admin users to download the approved degree letter in PDF format.
SRS-183	The system shall store all generated degree letters in a secure digital archive for future reference.
SRS-184	The system shall maintain version history for updated or corrected degree letters.

2.2.10.6 Process Degree Letter Reporting

SRS-185	The system shall allow admin users to generate reports of degree issuance requests based on department, program, supervisor, or year.
SRS-186	The system shall allow exporting degree issuance reports in PDF or Excel format.
SRS-187	The system shall track the total number of issued, pending, and rejected degree letters for statistical analysis.

2.3Non-Functional Requirements (NFRs)

2.3.1 Performance Requirements

The system should load all major pages (dashboard, records, submissions) within a few seconds under normal network conditions.

The system should handle multiple simultaneous users without performance degradation.

Search operations should return results quickly even when the database grows large.

2.3.2 Security Requirements

The system should protect user data through secure login, authentication, and encrypted password storage.

Sensitive documents such as thesis, synopsis, and reports should be stored securely and only accessible to authorized roles.

The system should ensure that user sessions automatically expire after a period of inactivity.

2.3.3 Usability Requirements

The system should provide a clean, simple, and user-friendly interface that is easy for students, supervisors, and admin to understand.

Important actions and pages (submit, upload, track status) should be easy to find and navigate.

Help messages, tooltips, and clear on-screen guidance should be available where needed.

2.3.4 Reliability Requirements

The system should be available and functional with minimal downtime.

All uploaded documents and stored data should remain safe and accessible at all times.

The system should recover gracefully from unexpected failures without losing data.

2.3.5 Portability Requirements

The system should be accessible from different browsers such as Chrome, Firefox, and Edge.

The web-based system should run smoothly on laptops, desktops, and mobile devices.

2.3.6 Scalability Requirements

The system should support an increasing number of users as the university grows or adds new campuses.

Additional modules and features should be easy to add without changing the core system.

2.3.7 Maintainability Requirements

The system should be easy to update, fix, and extend with new modules in the future.

Code and system architecture should be modular so that individual modules (thesis, progress reports, notifications) can be improved without affecting others.

2.3.8 Data Integrity Requirements

The system should ensure that stored information (student data, reports, thesis files) remains accurate, consistent, and free from corruption.

Duplicate data should be prevented through validation and strict input rules.

2.3.9 Backup & Recovery Requirements

The system should perform regular backups of all uploaded files and stored records.

The system should allow data restoration in case of accidental deletion or system failure.

2.3.10 Accessibility Requirements

The system should support accessibility features such as readable fonts, clear labels, and easy navigation for all user groups.

2.4 External Interface Requirements

2.4.1 User Interfaces

The GRMS will provide a responsive web-based user interface accessible from desktops, laptops, tablets, and mobile devices. Each user role—students, supervisors, and administrators—will have a customized dashboard displaying relevant tasks, notifications, and system updates. The interface will follow a consistent layout, color theme, and structured navigation menu to ensure ease of use. Forms such as synopsis submission, progress reports, thesis uploads, and meeting entries will include proper labels, input validation, and error messages. The goal of the UI is to offer a clean, intuitive, and user-friendly experience that supports efficient interaction with all system modules.

2.4.2 Hardware Interfaces

The GRMS will operate on standard computing hardware without requiring any specialized devices. Users can access the system through personal computers, laptops, or mobile devices connected to the internet. The server hosting the system must support sufficient storage to handle large files, such as thesis documents and evaluation reports. External devices, such as USB drives or scanners, may be used by users for uploading documents, but no direct hardware integration is needed. The system's hardware requirements are minimal and compatible with existing university infrastructure.

2.4.3 Software Interfaces

The GRMS will interact with multiple software components to provide smooth and secure functionality. It will connect to a relational database (such as MySQL or PostgreSQL) for storing all student, supervisor, and research records. The system will also integrate with an email service (SMTP) to send alerts, notifications, and verification messages. Web browsers such as Google Chrome, Microsoft Edge, Firefox, and Safari will serve as the primary platform for system access. Additionally, the system will support APIs, PDF generation tools, and authentication frameworks required for login, document handling, and communication between modules.

2.4.4 Communication Interfaces

The GRMS will use secure communication protocols to ensure safe data exchange between users and the server. All communication will be conducted through HTTPS to protect sensitive information such as login credentials and uploaded documents. The system will use SMTP for sending email notifications and may utilize RESTful APIs for data transfer between client and server modules. Data formats such as JSON will be used for structured communication. If real-time notifications are enabled, the system may also support WebSocket or similar technologies to deliver immediate updates to users

Chapter#4

Construction

Class Diagram

FOR

GRMS FOR FUUAST

VERSION 1.0

Prepared By

Alishba Arshad

Sana Tariq

10th dec, 2025

REVISION HISTORY

Version	Description	Author	Date
1.0	This covers the Class Diagram	Alishba Arshad and Sana Tariq	10th dec, 2025

4.1 Class Diagram:

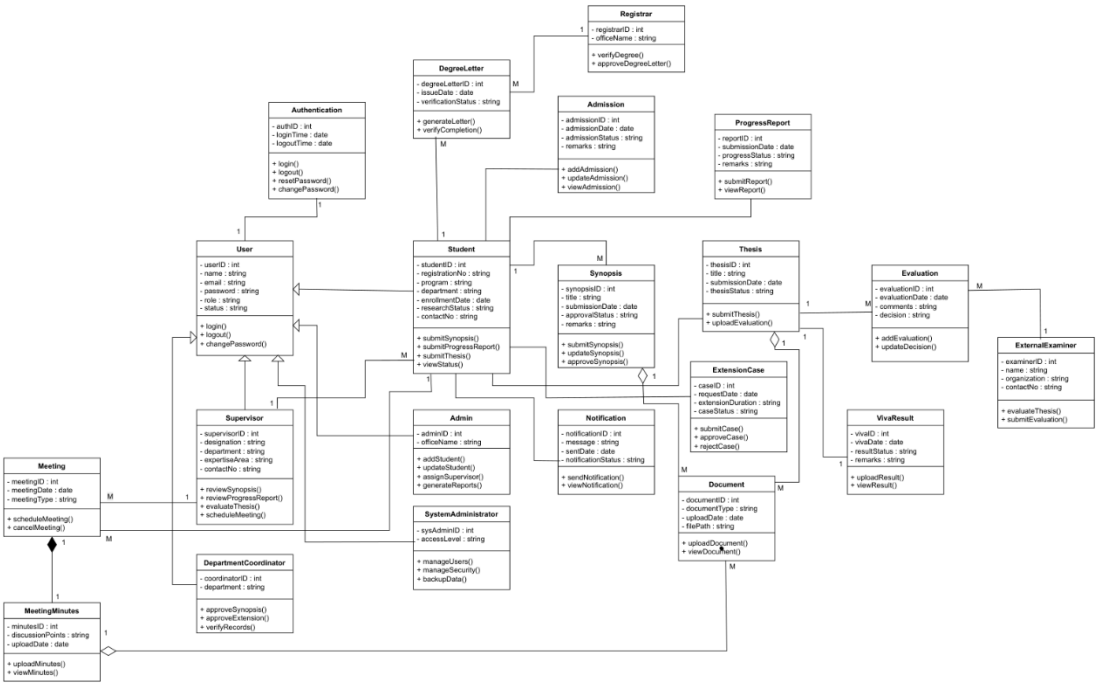

Figure 4.1: Class Diagram

4.2 Project Code:

Authentication Controller

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.models import User, Group
from django.contrib import messages

def login_view(request):
    """User login controller"""
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")

        user = authenticate(request, username=username, password=password)

        if user:
            login(request, user)

            if user.is_superuser or user.groups.filter(name='Admin').exists():
                return redirect('admin_dashboard')
            elif user.groups.filter(name='Supervisor').exists():
                return redirect('supervisor_dashboard')
            elif user.groups.filter(name='Examiner').exists():
```

```
            return redirect('examiner_dashboard')
        else:
            return redirect('student_dashboard')

    messages.error(request, "Invalid username or password")

return render(request, 'auth/login.html')



def register_view(request):
    """User registration controller"""
    if request.method == "POST":
        fullname = request.POST.get("fullname")
        username = request.POST.get("username")
        email = request.POST.get("email")
        role = request.POST.get("role")
        password = request.POST.get("password")
        confirm = request.POST.get("confirm_password")

        if password != confirm:
            messages.error(request, "Passwords do not match")
            return redirect("register")

        if User.objects.filter(username=username).exists():
            messages.error(request, "Username already exists")
            return redirect("register")
```

```
user = User.objects.create_user(
    username=username,
    email=email,
    password=password,
    first_name=fullname
)

group, _ = Group.objects.get_or_create(name=role)
user.groups.add(group)

if role == 'Student':
    Student.objects.create(
        user=user,
        registration_no=f"TEMP-{user.id}",
        department='Not Assigned',
        session='Not Assigned',
        program='Not Assigned',
        enrollment_status='Active',
        admission_date=timezone.now().date()
    )
elif role == 'Supervisor':
    Supervisor.objects.create(
        user=user,
        department='Not Assigned',
        designation='Not Assigned',
```

```
            availability_status='Available'
        )
    elif role == 'Examiner':
        Examiner.objects.create(
            user=user,
            name=fullname,
            email=email,
            phone_no='',
            designation='Not Assigned',
            examiner_type='Internal',
            institution='Not Assigned',
            area_of_expertise='Not Assigned',
            availability_status='Available'
        )

    messages.success(request, "Account created successfully. Please login.")
    return redirect("login")

return render(request, 'auth/register.html')


@login_required
def logout_view(request):
    """User logout controller"""
    logout(request)
    return redirect("login")
```

User Controller:

```
@login_required
@user_passes_test(is_admin)
def admin_student_list(request):
    """List all students"""
    students = Student.objects.select_related('user', 'supervisor').all()
    return render(request, 'admin/student/list.html', {'students': students})



@login_required
@user_passes_test(is_admin)
def admin_student_create(request):
    """Create new student"""
    if request.method == 'POST':
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        first_name = request.POST.get('first_name')
        last_name = request.POST.get('last_name')

        if User.objects.filter(username=username).exists():
            messages.error(request, 'Username already exists')
```

```
    return redirect('admin_student_create')

user = User.objects.create_user(
    username=username,
    email=email,
    password=password,
    first_name=first_name,
    last_name=last_name
)
group, _ = Group.objects.get_or_create(name='Student')
user.groups.add(group)

student = Student.objects.create(
    user=user,
    registration_no=request.POST.get('registration_no'),
    department=request.POST.get('department'),
    session=request.POST.get('session'),
    program=request.POST.get('program'),
    enrollment_status=request.POST.get('enrollment_status', 'Active'),
    admission_date=request.POST.get('admission_date') or timezone.now().date()
)

supervisor_id = request.POST.get('supervisor')
if supervisor_id:
    student.supervisor_id = supervisor_id
    student.save()
```

```
            messages.success(request, 'Student created successfully!')
            return redirect('admin_student_list')

        supervisors = Supervisor.objects.all()
        return render(request, 'admin/student/create.html', {'supervisors': supervisors})


@login_required
@user_passes_test(is_admin)
def admin_student_update(request, pk):
    """Update student"""
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        user = student.user
        user.first_name = request.POST.get('first_name', user.first_name)
        user.last_name = request.POST.get('last_name', user.last_name)
        user.email = request.POST.get('email', user.email)
        user.save()

        student.registration_no = request.POST.get('registration_no', student.registration_no)
        student.department = request.POST.get('department', student.department)
        student.session = request.POST.get('session', student.session)
        student.program = request.POST.get('program', student.program)
        student.enrollment_status = request.POST.get('enrollment_status',
student.enrollment_status)
```

```
        student.supervisor_id = request.POST.get('supervisor') or None
        student.save()

        messages.success(request, 'Student updated successfully!')
        return redirect('admin_student_list')

    supervisors = Supervisor.objects.all()
    return render(request, 'admin/student/update.html', {'student': student, 'supervisors':
supervisors})



@login_required
@user_passes_test(is_admin)
def admin_student_delete(request, pk):
    """Delete student"""
    student = get_object_or_404(Student, pk=pk)
    if request.method == 'POST':
        user = student.user
        student.delete()
        user.delete()
        messages.success(request, 'Student deleted successfully!')
        return redirect('admin_student_list')
    return render(request, 'admin/student/delete.html', {'student': student})

@login_required
@user_passes_test(is_admin)
```

```
def admin_supervisor_list(request):
    """List all supervisors"""
    supervisors = Supervisor.objects.select_related('user').all()
    return render(request, 'admin/supervisor/list.html', {'supervisors': supervisors})



@login_required
@user_passes_test(is_admin)
def admin_supervisor_create(request):
    """Create new supervisor"""
    if request.method == 'POST':
        username = request.POST.get('username')
        email = request.POST.get('email')
        password = request.POST.get('password')
        first_name = request.POST.get('first_name')
        last_name = request.POST.get('last_name')

        if User.objects.filter(username=username).exists():
            messages.error(request, f'Username "{username}" already exists.')
            return redirect('admin_supervisor_create')

        user = User.objects.create_user(
            username=username,
            email=email,
            password=password,
            first_name=first_name,
```

```
            last_name=last_name
        )
        group, _ = Group.objects.get_or_create(name='Supervisor')
        user.groups.add(group)

        emp_id = request.POST.get('employee_id')
        if emp_id == '' or emp_id == 'None' or emp_id is None:
            emp_id = None

        supervisor = Supervisor.objects.create(
            user=user,
            employee_id=emp_id,
            department=request.POST.get('department'),
            designation=request.POST.get('designation'),
            availability_status=request.POST.get('availability_status', 'Available'),
            max_students=int(request.POST.get('max_students', 5))
        )

        messages.success(request, 'Supervisor created successfully!')
        return redirect('admin_supervisor_list')

    return render(request, 'admin/supervisor/create.html')


@login_required
@user_passes_test(is_admin)
```

```
def admin_supervisor_update(request, pk):
    """Update supervisor"""
    supervisor = get_object_or_404(Supervisor, pk=pk)
    if request.method == 'POST':
        user = supervisor.user
        user.first_name = request.POST.get('first_name', user.first_name)
        user.last_name = request.POST.get('last_name', user.last_name)
        user.email = request.POST.get('email', user.email)
        user.save()

        supervisor.department = request.POST.get('department', supervisor.department)
        supervisor.designation = request.POST.get('designation', supervisor.designation)
        supervisor.availability_status = request.POST.get('availability_status',
supervisor.availability_status)
        supervisor.save()

        messages.success(request, 'Supervisor updated successfully!')
        return redirect('admin_supervisor_list')

    return render(request, 'admin/supervisor/update.html', {'supervisor': supervisor})



@login_required
@user_passes_test(is_admin)
def admin_supervisor_delete(request, pk):
    """Delete supervisor"""
```

```
    supervisor = get_object_or_404(Supervisor, pk=pk)
    if request.method == 'POST':
        user = supervisor.user
        supervisor.delete()
        user.delete()
        messages.success(request, 'Supervisor deleted successfully!')
        return redirect('admin_supervisor_list')
    return render(request, 'admin/supervisor/delete.html', {'supervisor': supervisor})
```

Admin Controller

```
@login_required
@user_passes_test(is_admin)
def admin_dashboard(request):
    """Admin dashboard controller"""
    total_students = Student.objects.count()
    active_students = Student.objects.filter(user__is_active=True).count()
    total_supervisors = Supervisor.objects.count()
    available_supervisors = Supervisor.objects.filter(availability_status='Available').count()
    pending_synopses = Synopsis.objects.filter(status='Pending').count()
    pending_theses = Thesis.objects.filter(status='Under Review').count()
    pending_extensions = ExtensionCase.objects.filter(status='Pending').count()

    recent_students = Student.objects.select_related('user').order_by('-admission_date')[:5]
```

```
    recent_synopses = Synopsis.objects.select_related('student').order_by('-
submission_date')[:5]
    recent_theses = Thesis.objects.select_related('student').order_by('-submission_date')[:5]

    context = {
        'total_students': total_students,
        'active_students': active_students,
        'total_supervisors': total_supervisors,
        'available_supervisors': available_supervisors,
        'pending_synopses': pending_synopses,
        'pending_theses': pending_theses,
        'pending_extensions': pending_extensions,
        'recent_students': recent_students,
        'recent_synopses': recent_synopses,
        'recent_theses': recent_theses,
    }
    return render(request, 'admin/dashboard.html', context)
```

Student Controller

```
@login_required
def student_dashboard(request):
    """Student dashboard controller"""
    try:
        student = Student.objects.get(user=request.user)
    except Student.DoesNotExist:
        messages.error(request, "Student profile not found.")
```

```
        return redirect('home')

    recent_synopses = Synopsis.objects.filter(student=student).order_by('-submission_date')[:5]
    recent_theses = Thesis.objects.filter(student=student).order_by('-submission_date')[:5]
    upcoming_meetings = Meeting.objects.filter(
        student=student,
        meeting_date__gte=timezone.now()
    ).order_by('meeting_date')[:5]
    total_reports = ProgressReport.objects.filter(student=student).count()

    context = {
        'student': student,
        'recent_synopses': recent_synopses,
        'recent_theses': recent_theses,
        'upcoming_meetings': upcoming_meetings,
        'total_reports': total_reports,
    }
    return render(request, 'student/dashboard.html', context)
```

Supervisor Controller

```
@login_required
def supervisor_dashboard(request):
    """Supervisor dashboard controller"""
```

```
    try:
        supervisor = Supervisor.objects.get(user=request.user)
    except Supervisor.DoesNotExist:
        messages.error(request, "Supervisor profile not found.")
        return redirect("home")

    assigned_students = Student.objects.filter(supervisor=supervisor)
    student_ids = assigned_students.values_list('id', flat=True)

    pending_synopses = Synopsis.objects.filter(supervisor=supervisor, status='Pending').count()
    pending_theses = Thesis.objects.filter(student__supervisor=supervisor, status='Under
Review').count()
    pending_reports = ProgressReport.objects.filter(student_id__in=student_ids,
status='Submitted').count()
    pending_extensions = ExtensionCase.objects.filter(student_id__in=student_ids,
status='Pending').count()

    upcoming_meetings = Meeting.objects.filter(
        supervisor=supervisor,
        meeting_date__gte=timezone.now()
    ).order_by('meeting_date')[:5]

    context = {
        'supervisor': supervisor,
        'total_students': assigned_students.count(),
        'pending_synopses': pending_synopses,
        'pending_theses': pending_theses,
```

```
        'pending_reports': pending_reports,
        'pending_extensions': pending_extensions,
        'upcoming_meetings': upcoming_meetings,
    }
    return render(request, 'supervisor/dashboard.html', context)
```

Synopsis Controller

```
@login_required
def student_synopsis_submit(request):
    """Student submit synopsis"""
    student = get_object_or_404(Student, user=request.user)
    if request.method == 'POST':
        form = SynopsisSubmitForm(request.POST, request.FILES)
        if form.is_valid():
            synopsis = form.save(commit=False)
            synopsis.student = student
            synopsis.status = 'submitted'
            synopsis.save()
            messages.success(request, "Synopsis submitted successfully!")
            return redirect('student_synopsis_list')
    else:
        form = SynopsisSubmitForm()
    return render(request, 'student/synopsis/submit.html', {'form': form})
```

```
@login_required
def supervisor_synopsis_review(request, pk):
    """Supervisor review synopsis"""
    synopsis = get_object_or_404(Synopsis, pk=pk)
    supervisor = get_object_or_404(Supervisor, user=request.user)

    if not (synopsis.supervisor == supervisor or synopsis.student.supervisor == supervisor):
        messages.error(request, "You are not authorized to review this synopsis.")
        return redirect('supervisor_dashboard')

    if request.method == 'POST':
        form = SynopsisReviewForm(request.POST, instance=synopsis)
        if form.is_valid():
            synopsis = form.save(commit=False)
            synopsis.reviewed_by = supervisor
            synopsis.review_date = timezone.now()
            synopsis.save()
            messages.success(request, "Review submitted.")
            return redirect('supervisor_synopsis_list')
    else:
        form = SynopsisReviewForm(instance=synopsis)
    return render(request, 'supervisor/synopsis/review.html', {'form': form, 'synopsis': synopsis})
```

4.3 Sample Queries:

As this project is developed using Django with its built-in ORM, direct SQL queries are rarely written. Instead, Django ORM provides a high-level, Pythonic way to interact with the database using Python methods.

1. Select and Find Queries

• Find by primary key: student = Student.objects.get(id=student_id)

• Find by unique field: user = User.objects.get(username=username)

• Find first record: synopsis = Synopsis.objects.filter(student=student).first()

• Find with related data: student = Student.objects.select_related('supervisor').get(id=pk)

• Multiple conditions: theses = Thesis.objects.filter(student=student, status='Submitted')

• Pagination with sorting: synopses = Synopsis.objects.all().order_by('-submission_date')[:5]

• Count records: total_students = Student.objects.count()

• Filter with Q objects: synopses = Synopsis.objects.filter(Q(status='Pending') | Q(status='Under Review'))

• Exclude records: active_students = Student.objects.exclude(enrollment_status='Inactive')

• Get latest record: latest_synopsis = Synopsis.objects.filter(student=student).latest('submission_date')

2. Create and Insert Queries

• Create a new record: student = Student.objects.create(user=user, registration_no='123', department='CS')

• Create with foreign key: synopsis = Synopsis.objects.create(student=student, title='Research', document=file)

• Create multiple records: Notification.objects.bulk_create([Notification(user=user1, message='Hi'), Notification(user=user2, message='Hello')])

• Get or create: group, created = Group.objects.get_or_create(name='Examiner')

• Update or create: student, created = Student.objects.update_or_create(user=user, defaults={'department': 'CS'})

3. Update and Modify Queries

• Update single record: synopsis.status = 'Approved' synopsis.save()

• Update multiple records: Synopsis.objects.filter(status='Pending').update(status='Under Review')

• Update with conditions: Student.objects.filter(enrollment_status='Inactive').update(enrollment_status='Active')

• Increment field: Student.objects.filter(id=student_id).update(attempts=F('attempts') + 1)

• Update timestamp: thesis.submission_date = timezone.now() thesis.save()

4. Delete and Remove Queries

• Delete single record: student.delete()

• Delete multiple records: Notification.objects.filter(is_read=True).delete()

• Delete with conditions: Synopsis.objects.filter(status='Rejected').delete()

• Delete all records: Notification.objects.all().delete()

5. Relationship Queries (Foreign Keys)

• Access related data (forward): student.supervisor.user.get_full_name()

• Access related data (reverse): supervisor.students.all()

• Filter by related field: synopses = Synopsis.objects.filter(student__supervisor=supervisor)

• Pre-fetch related data: students = Student.objects.prefetch_related('synopses').all()

• Select related (join): students = Student.objects.select_related('supervisor').all()

6. Aggregate and Annotate Queries

• Count with filter: pending_synopses = Synopsis.objects.filter(status='Pending').count()

• Sum of values: total_marks = ThesisEvaluation.objects.aggregate(Sum('marks'))

• Average: avg_marks = ThesisEvaluation.objects.aggregate(Avg('marks'))

• Group by with count: Student.objects.values('department').annotate(count=Count('id'))

• Group by with filter: Synopsis.objects.values('status').annotate(count=Count('id')).filter(count__gt=5)

Sample Queries in Django Shell

• Get all students with their supervisors: students = Student.objects.select_related('supervisor').all()

• Get pending synopses for a supervisor: pending_synopses = Synopsis.objects.filter(supervisor=supervisor, status='Pending')

• Get student's thesis with evaluations: thesis = Thesis.objects.filter(student=student).prefetch_related('evaluations').first()

• Get unread notifications for a user: notifications = Notification.objects.filter(user=request.user, is_read=False)

• Get total students count by department: department_counts = Student.objects.values('department').annotate(count=Count('id'))

• Get recent synopses (last 5): recent_synopses = Synopsis.objects.all().order_by('-submission_date')[:5]

• Update extension status: ExtensionCase.objects.filter(student=student, status='Pending').update(status='Approved')

• Delete old notifications (older than 30 days): cutoff_date = timezone.now() - timedelta(days=30) Notification.objects.filter(created_at__lt=cutoff_date).delete()

Chapter#5

Testing

Test Cases

FOR

GRMS FOR FUUAST

VERSION 1.0

Prepared By

Alishba Arshad

Sana Tariq

23rd june, 2026

REVISION HISTORY

VERSION	DESCRIPTION	AUTHOR	DATE
1.0	This is the Test case of the GRMS System	Alishba Arshad & Sana Tariq	23,June,2026

Test Cases

Test case ID:	TC-01
Test case Name:	Process Login
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a user Login into a system
Primary Actor:	<Admin><Student><Supervisor>

Main Success Scenario:

#	User Action	#	System Response
1.	User enters username/email and password		
2.	User selects role		
3.	User clicks Login		
		4.	System receives login request
		5.	System validates credentials
		6.	System grants access
		7.	System displays dashboard with success message

Testing Requirement:

Testing Condition:	The system must be running; The user must already be registered in the System database
Input Data:	enters username/email and password; selects role; clicks Login
Expected Result:	User session is created and dashboard is loaded
Actual Result:	User session is created and dashboard is loaded
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-02
Test case Name:	Change Password
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how a logged-in user updates their password
Primary Actor:	<Admin><Student><Supervisor>

Main Success Scenario:

#	User Action	#	System Response
1.	User selects Change Password		

		2.	System displays password form
3.	User enters old and new password		
4.	User confirms new password		
5.	User clicks Update		
		6.	System validates and updates password
		7.	System displays success message

Testing Requirement:

Testing Condition:	System must be running; User must be logged into System
Input Data:	selects Change Password; enters old and new password; clicks Update
Expected Result:	User successfully changed his/her password
Actual Result:	User successfully changed his/her password
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-03
Test case Name:	Forget Password
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025

Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how a user resets their forgotten password
Primary Actor:	<Admin>< Student>< Supervisor>

Main Success Scenario:

#	User Action	#	System Response
1.	User clicks "Forgot Password" link.		
		2.	System displays email input form.
3.	User enters registered email.		
4.	User clicks "Send Reset Link".		
		5.	System verifies email exists.
		6.	System sends password reset link to email.
7.	User clicks reset link in email.		
		8.	System opens password reset page.
9.	User enters new password.		
10.	User confirms new password.		
11.	User clicks "Reset Password".		
		12.	System updates password and displays "Password Reset Successfully."

Testing Requirement:

Testing Condition:	User must have a registered email address
Input Data:	clicks "Forgot Password" link.; enters registered email.; clicks "Send Reset Link".; clicks reset link in email.; enters new password.; clicks "Reset Password".
Expected Result:	User's password is updated successfully in the system
Actual Result:	User's password is updated successfully in the system
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-04	
Test case Name:	User Sign Up	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a new user creates an account in the system by providing required registration details	
Primary Actor:	<Admin>< Student>< Supervisor>	

Main Success Scenario:

#	User Action	#	System Response

1.	User opens the “Sign Up” page		
2.	2. System displays registration form		
3.	User enters required details		
4.	User clicks “Register”		
		5.	System validates entered information
		6.	System creates user account
		7.	System initiates account verification process
		8.	System displays “Registration Successful”

Testing Requirement:

Testing Condition:	System must be running; User must not already be registered
Input Data:	enters required details; clicks “Register”
Expected Result:	User account is created and pending verification
Actual Result:	User account is created and pending verification
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-05

Test case Name:	User Account Verification
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how a newly registered user verifies their account using email verification
Primary Actor:	<User>

Main Success Scenario:

#	User Action	#	System Response
1.	User opens verification email		
2.	User clicks verification link		
		3.	System validates verification token
		4.	System activates user account
		5.	System displays Account Verified Successfully

Testing Requirement:

Testing Condition:	User account is created; Verification email is sent
Input Data:	clicks verification link
Expected Result:	User account is verified and activated
Actual Result:	User account is verified and activated

Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-06
Test case Name:	Account Security Management
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the system enforces account security measures to protect user accounts
Primary Actor:	<System>

Main Success Scenario:

#	User Action	#	System Response
1.	User attempts login or system access		
		2.	System encrypts and validates credentials
		3.	System monitors failed login attempts
		4.	System enforces session timeout on inactivity
		5.	System logs security events

Testing Requirement:

Testing Condition:	User account exists
Input Data:	N/A
Expected Result:	Account security rules are enforced successfully
Actual Result:	Account security rules are enforced successfully
Priority:	High
Frequency:	Continuously
Test Acceptance:	Passed

Test case ID:	TC-07	
Test case Name:	Add Student	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how the admin adds a new student record into the system	
Primary Actor:	<Admin>	

Main Success Scenario:

#	User Action	#	System Response

1.	Admin opens the “Add Student” page		
		2.	System displays the student registration form
3.	Admin enters student details		
4.	Admin clicks the “Submit” button		
		5.	System validates the entered data
		6.	System checks for duplicate registration number or email
		7.	System saves the new student record in the database
		8.	System displays “Student Added Successfully”

Testing Requirement:

Testing Condition:	Admin is logged into the system; System is running.; Student record does not already exist in the database
Input Data:	enters student details; clicks the “Submit” button
Expected Result:	New student record is successfully stored in the system database
Actual Result:	New student record is successfully stored in the system database
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-08
Test case Name:	: Search Student
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how the admin searches for a student in the system
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin select student search page		
2.	Admin enters search criteria		
3.	Admin clicks the "Search" button		
		4.	System processes the search request
		5.	System retrieves matching student records from the database
		6.	System displays the search results to the admin

Testing Requirement:

Testing Condition:	Admin is logged into the system; Student records exist in the database

Input Data:	select student search page; enters search criteria; clicks the "Search" button
Expected Result:	Search results are displayed to the admin
Actual Result:	Search results are displayed to the admin
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-09	
Test case Name:	Update Student	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin updates a student's information in system	
Primary Actor:	<Admin>	

Main Success Scenario:

#	User Action	#	System Response
1.	Admin opens the "Update Student" page		
2.	Admin searches for the student using registration number		

3.	Admin clicks the "Search" button		
		4.	System retrieves the student record from the database
5.	Admin views student record		
		6.	System displays the student information
6.	Admin modifies the required fields (email, batch, program, etc.)		
7.	Admin clicks "Save" to update information		
		8.	System validates the updated data
		9.	System saves the changes and displays "Student Updated Successfully"

Testing Requirement:

Testing Condition:	Admin is logged into the system; The student record exists in the database
Input Data:	clicks the "Search" button; clicks "Save" to update information
Expected Result:	The student record is updated successfully in the system database
Actual Result:	The student record is updated successfully in the system database
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-10
Test case Name:	Assign Supervisor
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how the admin assigns a supervisor to a student
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin opens “Assign Supervisor” page		
2.	Admin selects a student		
		3.	System displays available supervisors
4.	Admin selects supervisor		
5.	Admin confirms assignment		
		6.	System saves assignment
		7.	System notifies student and supervisor

Testing Requirement:

Testing Condition:	Admin is logged in; Student record exists
Input Data:	selects a student; selects supervisor

Expected Result:	Supervisor is assigned successfully
Actual Result:	Supervisor is assigned successfully
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-11	
Test case Name:	View Enrollment Status	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how the enrollment status of a student	
Primary Actor:	<Student><Admin>	

Main Success Scenario:

#	User Action	#	System Response
1.	Student logs into the system		
2.	Student navigates to the Student Dashboard		
3.	Student selects the option “View Enrollment Status”		

		4.	System retrieves the student's enrollment details from the database
		5.	System displays enrollment status (e.g., Enrolled, Pending, Suspended), program, department, and session

Testing Requirement:

Testing Condition:	Student is logged into the system; Student record exists
Input Data:	selects the option "View Enrollment Status"
Expected Result:	Enrollment status is displayed to the student
Actual Result:	Enrollment status is displayed to the student
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-12	
Test case Name:	Export Student Data	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin exports student data from the system in a supported format such as PDF or Excel	

Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin opens the Student Management module		
2.	Admin selects the “Export Student Data” option		
		3.	System displays available export formats
4.	Admin selects the required format		
5.	Admin clicks the “Export” button		
		6.	System retrieves student records
		7.	System generates the export file
		8.	System provides the download link
9.	Admin downloads the exported file		

Testing Requirement:

Testing Condition:	Admin is logged into the system; Student records exist in the system database
Input Data:	selects the “Export Student Data” option; selects the required format; clicks the “Export” button
Expected Result:	Student data is successfully exported

Actual Result:	Student data is successfully exported
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-13
Test case Name:	Submit Synopsis
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a student submits their research synopsis to the system for supervisor review
Primary Actor:	<Student>

Main Success Scenario:

#	User Action	#	System Response
1.	Student select submit synopsis		
		2.	System displays synopsis submission form
3.	Student enters synopsis details		
4.	Student uploads synopsis document		

		5.	System validates the uploaded file
		6.	System saves synopsis information
		7.	System sends notification to supervisor
		8.	System displays submission confirmation

Testing Requirement:

Testing Condition:	Student must be logged in; Student must be registered in the system; Supervisor must be assigned to the student
Input Data:	select submit synopsis; enters synopsis details; uploads synopsis document
Expected Result:	Synopsis is successfully submitted and marked as Pending Approval
Actual Result:	Synopsis is successfully submitted and marked as Pending Approval
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-14	
Test case Name:	Update Synopsis	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025

Test case Description:	This use case describes how a student updates a previously submitted synopsis in the system after receiving feedback or before final approval
Primary Actor:	<Student>

Main Success Scenario:

#	User Action	#	System Response
1.	Student selects submitted synopsis		
		2.	System displays synopsis details
3.	Student edits synopsis information		
		5.	System validates updated file
		6.	System saves updated synopsis
		7.	System notifies supervisor
		8.	System shows update confirmation

Testing Requirement:

Testing Condition:	Student must be logged in; Synopsis must already be submitted; Synopsis must not be finally approved
Input Data:	selects submitted synopsis
Expected Result:	Synopsis is successfully updated and marked as Re-Submitted
Actual Result:	Synopsis is successfully updated and marked as Re-Submitted
Priority:	High

Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-15	
Test case Name:	Review Synopsis	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a supervisor reviews a student's submitted synopsis in the system	
Primary Actor:	<Supervisor>	

Main Success Scenario:

#	User Action	#	System Response
1.	Supervisor selects view synopsis		
		2.	System displays submitted synopsis
3.	Supervisor reviews synopsis details		
		4.	System allows supervisor to add comments
5.	Supervisor submits review feedback		
		6.	System saves review comments

		7.	System notifies student

Testing Requirement:

Testing Condition:	Supervisor must be logged in; Synopsis must be submitted by student; Supervisor must be assigned to the student
Input Data:	selects view synopsis
Expected Result:	Synopsis review is completed and feedback is recorded in the system
Actual Result:	Synopsis review is completed and feedback is recorded in the system
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-16
Test case Name:	Approve Synopsis
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the supervisor approves a student's submitted synopsis in the system
Primary Actor:	<Supervisor>

Main Success Scenario:

#	User Action	#	System Response
1.	Supervisor select student synopsis		
		2.	System displays synopsis details
3.	Supervisor reviews synopsis		
4.	Supervisor selects “Approve”		
		5.	System updates synopsis status
		6.	System saves approval record
		7.	System notifies student
		8.	System displays approval confirmation

Testing Requirement:

Testing Condition:	Supervisor must be logged in; Synopsis must be submitted or revised; Supervisor must be assigned to the student
Input Data:	select student synopsis; selects “Approve”
Expected Result:	Synopsis status is updated to Approved
Actual Result:	Synopsis status is updated to Approved
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-17
Test case Name:	Reject Synopsis
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the supervisor rejects a student's submitted synopsis when it does not meet the required research criteria
Primary Actor:	<Supervisor>

Main Success Scenario:

#	User Action	#	System Response
1.	Supervisor select student synopsis		
		2.	System displays synopsis details
3.	Supervisor reviews synopsis		
4.	Supervisor selects "Reject"		
5.	Supervisor enters rejection comments		
		6.	System saves rejection decision
		7.	System updates synopsis status
		8.	System notifies student
		9.	System displays rejection confirmation

Testing Requirement:

Testing Condition:	Supervisor must be logged in; Synopsis must be submitted or revised; Supervisor must be assigned to the student
Input Data:	select student synopsis; selects “Reject”; enters rejection comments
Expected Result:	Synopsis status is updated to Rejected, and feedback is stored
Actual Result:	Synopsis status is updated to Rejected, and feedback is stored
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-18	
Test case Name:	View Synopsis Status	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a student views the current status of their submitted synopsis in the system	
Primary Actor:	<Supervisor>	

Main Success Scenario:

#	User Action	#	System Response
1.	Student selects “View Synopsis Status		
		2.	System retrieves the synopsis record
3.	Student requests status		
		4.	System displays current status
5.	Student optionally views supervisor comments		
		6.	System displays comments if available

Testing Requirement:

Testing Condition:	Student must be logged in; Synopsis must have been submitted
Input Data:	selects “View Synopsis Status
Expected Result:	Student sees the latest synopsis status and any supervisor feedback
Actual Result:	Student sees the latest synopsis status and any supervisor feedback
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-19
Test case Name:	Synopsis Reporting

Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin generates reports related to student synopses, including submission status, approval status, and supervisor-wise records
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects “Synopsis Reporting”		
		2.	System displays reporting options
3.	Admin selects report criteria		
		4.	System retrieves synopsis data
		5.	System generates synopsis report
		6.	System displays report results

Testing Requirement:

Testing Condition:	Admin must be logged in; Synopsis records must exist in the system
Input Data:	selects “Synopsis Reporting”; selects report criteria
Expected Result:	Synopsis report is successfully generated and available for viewing or export
Actual Result:	Synopsis report is successfully generated and available for viewing or export

Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-20	
Test case Name:	Upload Revised Synopsis	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a student uploads a revised version of the synopsis after receiving feedback from the supervisor	
Primary Actor:	<Student>	

Main Success Scenario:

#	User Action	#	System Response
1.	Student select revised synopsis file		
		2.	System displays revised upload form
3.	Student uploads revised synopsis file		
		4.	System validates file format and size
		5.	System saves revised synopsis

		6.	System updates synopsis status
		7.	System notifies supervisor
		8.	System displays upload confirmation

Testing Requirement:

Testing Condition:	Student must be logged in; Original synopsis must be submitted; Supervisor feedback must exist; Synopsis must not be finally approved
Input Data:	select revised synopsis file; uploads revised synopsis file
Expected Result:	Revised synopsis is successfully uploaded and marked as Pending Re-Review
Actual Result:	Revised synopsis is successfully uploaded and marked as Pending Re-Review
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-21	
Test case Name:	View Student Profile	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how an admin or supervisor views a student's profile in the system	

Primary Actor:	<Admin><Supervisor>

Main Success Scenario:

#	User Action	#	System Response
1.	User searches for student		
		2.	System retrieves student profile
		3.	System displays student details

Testing Requirement:

Testing Condition:	User must be logged in; Student record must exist
Input Data:	N/A
Expected Result:	Student profile is displayed successfully
Actual Result:	Student profile is displayed successfully
Priority:	Medium
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-22
Test case Name:	Update Student Profile

Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how the admin updates a student's personal or academic profile
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects student profile		
		2.	System displays editable form
3.	Admin updates information		
		4.	System validates data
		5.	System saves changes
		6.	System confirms update

Testing Requirement:

Testing Condition:	Admin must be logged in; Student record must exist
Input Data:	selects student profile
Expected Result:	Student profile is updated successfully
Actual Result:	Student profile is updated successfully

Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-23
Test case Name:	Manage Academic Records
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin manages a student's academic records in system
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects academic record option		
		2.	System displays academic data
3.	Admin updates academic details		
		4.	System saves academic records

Testing Requirement:

Testing Condition:	Student must be registered in the system
Input Data:	selects academic record option
Expected Result:	Academic records are updated successfully
Actual Result:	Academic records are updated successfully
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-24	
Test case Name:	Update Student Status	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how the admin updates a student's status (active, completed, dropped)	
Primary Actor:	<Admin>	

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects student		

2.	Admin updates status		
		3.	System saves updated status
		4.	System confirms change

Testing Requirement:

Testing Condition:	1.Student record must exist
Input Data:	selects student
Expected Result:	Student status is updated
Actual Result:	Student status is updated
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-25	
Test case Name:	Search Student	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin searches for a student in the system using basic search criteria	

Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin enters student ID or name		
		2.	System searches student records
		3.	System displays matching student details

Testing Requirement:

Testing Condition:	1. Admin must be logged in; 2. Student records must exist
Input Data:	enters student ID or name
Expected Result:	Requested student record is displayed successfully
Actual Result:	Requested student record is displayed successfully
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-26
Test case Name:	Filter Student

Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin filters student records based on selected criteria such as status, department, or supervisor
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects filter criteria		
		2.	System applies filters
		3.	System displays filtered student list

Testing Requirement:

Testing Condition:	1.Admin must be logged in; 2.Student records must exist
Input Data:	selects filter criteria
Expected Result:	Filtered student records are displayed successfully
Actual Result:	Filtered student records are displayed successfully
Priority:	Medium
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-27
Test case Name:	Manage Research Progress
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the supervisor manages and updates the research progress of an assigned student in the system
Primary Actor:	<Supervisor>

Main Success Scenario:

#	User Action	#	System Response
1.	Supervisor selects assigned student		
		2.	System displays student research progress details
3.	Supervisor updates progress status/comments		
		4.	System validates entered information
		5.	System saves updated progress record
		6.	System displays confirmation message
		7.	System notifies student

Testing Requirement:

Testing Condition:	1.Supervisor must be logged in; 2.Student must be assigned to the supervisor; 3.Student research record must exist
Input Data:	selects assigned student
Expected Result:	Student research progress is successfully updated and stored in the system
Actual Result:	Student research progress is successfully updated and stored in the system
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-28	
Test case Name:	Add Supervisor Record	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin adds a new supervisor record into the system	
Primary Actor:	<Admin>	

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects “Add Supervisor”		

		2.	System displays supervisor registration form
3.	Admin enters supervisor details		
		4.	System validates entered information
		5.	System saves supervisor record
		6.	System displays confirmation message

Testing Requirement:

Testing Condition:	1.Admin must be logged in; 2. System must be operational
Input Data:	selects “Add Supervisor”; enters supervisor details
Expected Result:	Supervisor record is successfully created in the system
Actual Result:	Supervisor record is successfully created in the system
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-29	
Test case Name:	Update Supervisor Record	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025

Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how the admin updates an existing supervisor's information
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin searches supervisor		
		2.	System displays supervisor details
3.	Admin updates information		
		4.	System validates updated data
		5.	System saves changes
		6.	System confirms update

Testing Requirement:

Testing Condition:	1.Admin must be logged in; 2. Supervisor record must exist
Input Data:	N/A
Expected Result:	Supervisor information is updated successfully
Actual Result:	Supervisor information is updated successfully
Priority:	Medium
Frequency:	Less Frequently Use

Test Acceptance:	Passed

Test case ID:	TC-30	
Test case Name:	View Supervisor Record	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin or department views supervisor details in system	
Primary Actor:	<Admin>	

Main Success Scenario:

#	User Action	#	System Response
1.	Admin searches supervisor		
		2.	System retrieves supervisor record
		3.	System displays supervisor details

Testing Requirement:

Testing Condition:	1. Admin must be logged in
Input Data:	N/A
Expected Result:	Supervisor record is displayed successfully

Actual Result:	Supervisor record is displayed successfully
Priority:	Medium
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-31	
Test case Name:	Assign Supervisor	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin assigns a supervisor to a student in the system	
Primary Actor:	<Admin>	

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects student		
		2.	System displays available supervisors
3.	3. Admin assigns supervisor		
		4.	System saves assignment

		5.	System confirms assignment

Testing Requirement:

Testing Condition:	Student and supervisor records must exist
Input Data:	selects student
Expected Result:	Supervisor is successfully assigned to the student
Actual Result:	Supervisor is successfully assigned to the student
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-32
Test case Name:	Manage Supervisor Availability
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how the admin manages supervisor availability status for student supervision
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	1. Admin selects supervisor		
2.	Admin updates availability status		
		3.	System saves availability information
		4.	System confirms update

Testing Requirement:

Testing Condition:	1.Supervisor record must exist
Input Data:	1. Admin selects supervisor
Expected Result:	Supervisor availability status is updated successfully
Actual Result:	Supervisor availability status is updated successfully
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-33	
Test case Name:	Supervisor Reporting	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025

Test case Description:	This use case describes how the admin generates reports related to supervisors and their assigned students
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects supervisor reporting		
		2.	System displays report options
3.	Admin selects report criteria		
		4.	System generates report
		5.	System displays report

Testing Requirement:

Testing Condition:	1.Supervisor records must exist
Input Data:	selects supervisor reporting; selects report criteria
Expected Result:	Supervisor report is generated successfully
Actual Result:	Supervisor report is generated successfully
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-34
Test case Name:	Supervisor Record Management
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin manages supervisor records, including viewing and maintaining supervisor data
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin accesses supervisor records		
		2.	System displays supervisor list
3.	Admin manages supervisor information		
		4.	System saves updates
		5.	System confirms changes

Testing Requirement:

Testing Condition:	Admin must be logged in
Input Data:	N/A

Expected Result:	Supervisor records are maintained successfully
Actual Result:	Supervisor records are maintained successfully
Priority:	Medium
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-35	
Test case Name:	Submit Progress Report	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a student uploads their progress report in the system	
Primary Actor:	<Student>	

Main Success Scenario:

#	User Action	#	System Response
1.	Student opens "Submit Progress Report" page		
2.	Student enters details		
3.	Student uploads PDF progress report		

		4.	System validates file format
5.	Student clicks "Submit"		
		6.	System stores the report in the database
		7.	System sends notification to supervisor
		8.	System shows "Progress Report Submitted Successfully"

Testing Requirement:

Testing Condition:	Student is logged into the system; Supervisor is assigned to student
Input Data:	enters details; uploads PDF progress report; clicks "Submit"
Expected Result:	Progress report stored and ready for review
Actual Result:	Progress report stored and ready for review
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-36	
Test case Name:	Review Progress Report	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025

Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how Supervisor reviews and evaluates submitted progress report
Primary Actor:	<Supervisor>

Main Success Scenario:

#	User Action	#	System Response
1.	Supervisor opens "Review Progress Report" page		
		2.	System displays list of pending reports
3.	Supervisor reviews submitted report details		
		4.	System loads report details and attached file
5.	Supervisor reviews content and adds comments/feedback		
		6.	System saves comments in draft state
7.	Supervisor selects evaluation status		
		8.	System updates report status
9.	Supervisor clicks "Submit Evaluation"		
		10.	System saves evaluation, notifies student, and updates dashboard

		11.	System displays "Evaluation Submitted Successfully"

Testing Requirement:

Testing Condition:	Supervisor is logged into the system; A progress report has been submitted by the student
Input Data:	selects evaluation status; clicks "Submit Evaluation"
Expected Result:	Progress report status is updated and student is notified
Actual Result:	Progress report status is updated and student is notified
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-37
Test case Name:	Upload Revised Progress Report
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a student uploads a revised progress report after receiving feedback
Primary Actor:	<Student>

Main Success Scenario:

#	User Action	#	System Response
1.	Student selects revised report option		
		2.	System displays upload form
3.	Student uploads revised report		
		4.	System validates file
		5.	System saves revised report
		6.	System notifies supervisor

Testing Requirement:

Testing Condition:	Student must be logged in; Progress report must already be submitted; Feedback must exist
Input Data:	selects revised report option; uploads revised report
Expected Result:	Revised progress report is saved and marked Pending Review
Actual Result:	Revised progress report is saved and marked Pending Review
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-38

Test case Name:	Track Report Status
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how a student tracks the status of submitted progress reports
Primary Actor:	<Student>

Main Success Scenario:

#	User Action	#	System Response
1.	Student selects “Track Report Status”		
		2.	System retrieves report status
		3.	System displays current status

Testing Requirement:

Testing Condition:	1.Student must be logged in
Input Data:	selects “Track Report Status”
Expected Result:	Report status is displayed
Actual Result:	Report status is displayed
Priority:	Medium
Frequency:	Frequently Use

Test Acceptance:	Passed

Test case ID:	TC-39
Test case Name:	Progress Evaluation
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the supervisor evaluates a student's progress report
Primary Actor:	<Supervisor>

Main Success Scenario:

#	User Action	#	System Response
1.	Supervisor selects progress report		
		2.	System displays report
3.	Supervisor evaluates and adds remarks		
		4.	System saves evaluation
		5.	System updates report status
		6.	System notifies student

Testing Requirement:

Testing Condition:	Supervisor must be logged in; Progress report must be submitted
Input Data:	selects progress report
Expected Result:	Progress report is evaluated and recorded
Actual Result:	Progress report is evaluated and recorded
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-40
Test case Name:	Progress Reporting
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin generates reports related to student progress reports
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects progress reporting		

		2.	System displays report options
3.	Admin selects criteria		
		4.	System generates report
		5.	System displays report

Testing Requirement:

Testing Condition:	Progress report records must exist
Input Data:	selects progress reporting; selects criteria
Expected Result:	Progress report is generated successfully
Actual Result:	Progress report is generated successfully
Priority:	High
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-41	
Test case Name:	Progress Record Management	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin manages progress report record	

Primary Actor:	<Student>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects progress reporting		
		2.	System displays report options
3.	Admin selects criteria		
		4.	System generates report
		5.	System displays report

Testing Requirement:

Testing Condition:	Progress report records must exist
Input Data:	selects progress reporting; selects criteria
Expected Result:	Progress report is generated successfully
Actual Result:	Progress report is generated successfully
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-42

Test case Name:	Submit Thesis
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how a student submits their thesis for evaluation
Primary Actor:	<Student>

Main Success Scenario:

#	User Action	#	System Response
1.	Student opens "Submit Thesis" page.		
		2.	System displays thesis submission form.
3.	Student enters thesis details (title, abstract, keywords).		
		4.	System validates input in real-time.
5.	Student uploads thesis PDF file.		
		6.	System validates file format and size.
7.	Student clicks "Submit Thesis".		
		8.	System saves thesis and notifies supervisor and admin.
		9.	System displays "Thesis Submitted Successfully."

Testing Requirement:

Testing Condition:	Student is logged in.; Student has completed all required milestones (synopsis approval, progress reports)
Input Data:	enters thesis details (title, abstract, keywords).; uploads thesis PDF file.; clicks "Submit Thesis".
Expected Result:	Thesis is stored in system and ready for examiner assignment
Actual Result:	Thesis is stored in system and ready for examiner assignment
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-43
Test case Name:	Assign Thesis Examiners
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how admin assigns internal/external examiners for thesis evaluation
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin opens "Assign Examiners" page		
		2.	System displays list of submitted theses
3.	Admin selects a thesis and clicks "Assign Examiners"		
		4.	System shows list of available examiners
5.	Admin selects internal and external examiners		
		6.	System validates availability and conflict
7.	Admin clicks "Confirm Assignment"		
		8.	System saves assignment and notifies examiners via email
		9.	System displays "Examiners Assigned Successfully"

Testing Requirement:

Testing Condition:	Admin is logged in; Thesis has been submitted by student
Input Data:	selects a thesis and clicks "Assign Examiners"; selects internal and external examiners; clicks "Confirm Assignment"
Expected Result:	Examiners are assigned and notified
Actual Result:	Examiners are assigned and notified
Priority:	High

Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-44	
Test case Name:	Upload Examiner Reports	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how examiners upload their evaluation reports	
Primary Actor:	<Examiner>	

Main Success Scenario:

#	User Action	#	System Response
1.	Examiner opens My Assignments page		
		2.	System displays assigned thesis
3.	Examiner selects a thesis and clicks "Upload Report"		
		4.	System opens upload form
5.	Examiner uploads evaluation report (PDF)		
		6.	System validates file

7.	Examiner enters comments and score		
		8.	System saves draft temporarily
9.	Examiner clicks "Submit Report"		
		10.	System saves data temporarily
		11.	System displays "Report Submitted Successfully"

Testing Requirement:

Testing Condition:	Examiner is logged in; Examiner has been assigned a thesis
Input Data:	Examiner selects a thesis and clicks "Upload Report"; Examiner uploads evaluation report (PDF); Examiner enters comments and score; Examiner clicks "Submit Report"
Expected Result:	Examiner report is stored and available for review
Actual Result:	Examiner report is stored and available for review
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-45
Test case Name:	Thesis Evaluation Workflow

Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes the end-to-end thesis evaluation workflow	
Primary Actor:	<Admi><Supervisor><Examiner>	

Main Success Scenario:

#	User Action	#	System Response
1.	Admin reviews examiner reports.		
		2.	System consolidates reports.
3.	Admin marks thesis status		
		4.	System updates status and notifies student.
5.	If changes required, student revises and resubmits.		
		6.	System tracks revisions.
7.	Admin finalizes evaluation and approves thesis.		
		8.	System updates final status and archives thesis.
		9.	System notifies all stakeholders.

Testing Requirement:

Testing Condition:	Thesis is submitted; Examiner reports are uploaded
Input Data:	N/A
Expected Result:	Thesis evaluation is complete and decision is recorded
Actual Result:	Thesis evaluation is complete and decision is recorded
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-46	
Test case Name:	Upload Revised Thesis	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how a student uploads a revised thesis after feedback	
Primary Actor:	<Student>	

Main Success Scenario:

#	User Action	#	System Response
1.	Student opens "My Thesis" page		

		2.	System displays current thesis status and feedback
3.	Student clicks "Upload Revised Thesis"		
		4.	System opens upload form
5.	Student uploads revised PDF		
		6.	System validates file and stores as new version
7.	Student clicks "Submit Revision"		
		8.	System notifies examiners and updates version history
		9.	System displays "Revised Thesis Submitted Successfully"

Testing Requirement:

Testing Condition:	Student is logged in; Thesis requires revisions based on examiner feedback
Input Data:	clicks "Upload Revised Thesis"; uploads revised PDF; clicks "Submit Revision"
Expected Result:	Revised thesis is submitted and examiners are notified
Actual Result:	Revised thesis is submitted and examiners are notified
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-47
Test case Name:	Generate Thesis Evaluation Summary
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the system generates a thesis evaluation summary report
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin opens "Reports" section		
		2.	System displays report generation options
3.	Admin selects "Thesis Evaluation Summary”		
		4.	System shows filters (department, date, status)
5.	Admin applies filters and clicks "Generate"		
		6.	System compiles data and generates PDF/Excel report
7.	Admin downloads or exports report		

		8.	System provides download link and logs activity
		9.	System archives generated report

Testing Requirement:

Testing Condition:	Thesis evaluation is complete; Admin is logged in
Input Data:	selects "Thesis Evaluation Summary"; applies filters and clicks "Generate"
Expected Result:	Summary report is generated and archived
Actual Result:	Summary report is generated and archived
Priority:	Medium
Frequency:	Occasionally Use
Test Acceptance:	Passed

Test case ID:	TC-48	
Test case Name:	Schedule Meeting	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how a supervisor schedules a meeting with a student	
Primary Actor:	<Supervisor>	

Main Success Scenario:

#	User Action	#	System Response
1.	Supervisor opens "Schedule Meeting" page.		
		2.	System displays meeting scheduling form.
3.	Supervisor selects student from list.		
		4.	System loads student details.
5.	Supervisor sets meeting date, time, mode (online/physical), and agenda.		
		6.	System validates date/time.
7.	Supervisor clicks "Schedule Meeting".		
		8.	System saves meeting, notifies student via email/system notification.
		9.	System displays "Meeting Scheduled Successfully."

Testing Requirement:

Testing Condition:	Supervisor is logged into the system; Student is assigned to the supervisor
Input Data:	selects student from list.; clicks "Schedule Meeting".
Expected Result:	Meeting is scheduled and both parties are notified
Actual Result:	Meeting is scheduled and both parties are notified
Priority:	Medium

Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-49	
Test case Name:	Record Meeting Details	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a supervisor records meeting minutes and details after a meeting	
Primary Actor:	<Supervisor>	

Main Success Scenario:

#	User Action	#	System Response
1.	Supervisor opens "Meeting Records" page.		
		2.	System shows list of past meetings
3.	Supervisor selects a meeting to update		
		4.	System loads meeting details
5.	Supervisor enters summary, tasks assigned, and progress discussed		
		6.	System auto-saves draft

7.	Supervisor uploads supporting files if any		
		8.	System validates file format and size.
		9s.	System stores record with timestamp and notifies student

Testing Requirement:

Testing Condition:	Supervisor is logged in; A meeting has been scheduled and conducted
Input Data:	selects a meeting to update; enters summary, tasks assigned, and progress discussed; uploads supporting files if any
Expected Result:	Meeting details are stored and accessible
Actual Result:	Meeting details are stored and accessible
Priority:	Medium
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-50	
Test case Name:	View Meeting History	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how users view past meeting records	

Primary Actor:	<Student><Supervisor><Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	User opens "Meeting History" page.		
		2.	System displays filter options (date, student, topic)
3.	User applies filters if needed		
		4.	System updates list in real-time
5.	User selects a meeting record		
		6.	System shows full details (summary, tasks, files)
7.	User may download attached files.		
		8.	System allows download
		9.	System logs the view activity

Testing Requirement:

Testing Condition:	User is logged in; Meeting records exist in the system
Input Data:	selects a meeting record
Expected Result:	User views meeting history successfully
Actual Result:	User views meeting history successfully

Priority:	Low
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-51
Test case Name:	Update Meeting Records
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the supervisor updates meeting records with students in the system.
Primary Actor:	<Supervisor>

Main Success Scenario:

#	User Action	#	System Response
1.	Supervisor selects meeting record		
		2.	System displays meeting details
3.	Supervisor updates meeting notes		
		4.	System validates information
		5.	System saves updated meeting record

		6.	System confirms update

Testing Requirement:

Testing Condition:	Supervisor must be logged in; Meeting record must exist
Input Data:	selects meeting record
Expected Result:	Meeting record is updated successfully.
Actual Result:	Meeting record is updated successfully.
Priority:	Medium
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-52
Test case Name:	Meeting Reminders
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the system sends meeting reminders to students and supervisors
Primary Actor:	<System>

Main Success Scenario:

#	User Action	#	System Response
		1.	System detects upcoming meeting
		2.	System generates reminder notification
		3.	System sends reminder to participants

Testing Requirement:

Testing Condition:	Meeting must be scheduled.
Input Data:	N/A
Expected Result:	Meeting reminder is successfully sent
Actual Result:	Meeting reminder is successfully sent
Priority:	Low
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-53	
Test case Name:	Meeting Report Management	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025

Test case Description:	This use case describes how the admin generates reports related to student-supervisor meetings
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects meeting reporting		
		2.	System displays report options
3.	Admin selects criteria		
		4.	System generates report
		5.	System displays report

Testing Requirement:

Testing Condition:	.Meeting records must exist.
Input Data:	selects meeting reporting; selects criteria
Expected Result:	Meeting report is generated successfully
Actual Result:	Meeting report is generated successfully
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-54
Test case Name:	Meeting Record Management
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin manages meeting records in the GRMS system
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin accesses meeting records		
		2.	System displays meeting list
3.	Admin manages meeting records		
		4.	System saves updates

Testing Requirement:

Testing Condition:	1. Admin must be logged in.
Input Data:	N/A
Expected Result:	Meeting records are maintained successfully

Actual Result:	Meeting records are maintained successfully
Priority:	Medium
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-55	
Test case Name:	Send System Notifications	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the system sends automated notifications for events	
Primary Actor:	<System>	

Main Success Scenario:

#	User Action	#	System Response

Testing Requirement:

Testing Condition:	System is running; Notification triggers are configured
Input Data:	N/A
Expected Result:	Notification is delivered and logged

Actual Result:	Notification is delivered and logged
Priority:	High
Frequency:	Continuously
Test Acceptance:	Passed

Test case ID:	TC-56
Test case Name:	Send Email Alerts
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how the system sends email alerts for important updates
Primary Actor:	<System>

Main Success Scenario:

#	User Action	#	System Response
		1.	System event triggers an email alert
		2.	System Retrieve recipient email & template
		3.	System populates Populate template with dynamic data

4.	Send email via SMTP		
		5.	System Confirm delivery & log activity
		6.	System Update notification status as “Sent”
		7.	System Archive sent email for reference

Testing Requirement:

Testing Condition:	Email service is configured
Input Data:	N/A
Expected Result:	Email is sent and delivery is confirmed
Actual Result:	Email is sent and delivery is confirmed
Priority:	High
Frequency:	Frequently
Test Acceptance:	Passed

Test case ID:	TC-57	
Test case Name:	Deadline and Reminders	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025

Test case Description:	This use case describes how the system generates and sends deadline notifications and reminders to users
Primary Actor:	<System>

Main Success Scenario:

#	User Action	#	System Response
		1.	System identifies upcoming deadlines
		2.	System generates reminder notifications
		3.	System sends notifications to users

Testing Requirement:

Testing Condition:	Relevant deadlines must be defined in the system
Input Data:	N/A
Expected Result:	Deadline reminders are successfully sent
Actual Result:	Deadline reminders are successfully sent
Priority:	High
Frequency:	Frequently used
Test Acceptance:	Passed

Test case ID:	TC-58

Test case Name:	Notification Dashboard
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how users view all system notifications in a centralized dashboard
Primary Actor:	<Student> <Supervisor> <Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	User opens notification dashboard		
		2.	System retrieves notifications
		3.	System displays notification list

Testing Requirement:

Testing Condition:	User must be logged in
Input Data:	N/A
Expected Result:	Notifications are displayed to the user
Actual Result:	Notifications are displayed to the user
Priority:	Medium
Frequency:	Frequently used

Test Acceptance:	Passed

Test case ID:	TC-59
Test case Name:	Notification Logging and Tracking
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the system logs and tracks sent notifications for monitoring purposes
Primary Actor:	<System>

Main Success Scenario:

#	User Action	#	System Response
		1.	System logs notification details
		2.	System tracks delivery status
		3.	System stores log information

Testing Requirement:

Testing Condition:	Notifications must be generated.
Input Data:	N/A
Expected Result:	Notification logs are stored successfully

Actual Result:	Notification logs are stored successfully
Priority:	Medium
Frequency:	Less Frequently used
Test Acceptance:	Passed

Test case ID:	TC-60
Test case Name:	Notification Setting Management
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how users manage their notification preferences in the system
Primary Actor:	<Student> <Supervisor> <Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	User opens notification settings		
		2.	System displays current preferences
3.	User updates notification options		
		4.	System saves updated settings

		5.	System confirms changes

Testing Requirement:

Testing Condition:	User must be logged in
Input Data:	N/A
Expected Result:	Notification settings are updated successfully
Actual Result:	Notification settings are updated successfully
Priority:	Medium
Frequency:	Less Frequently used
Test Acceptance:	Passed

Test case ID:	TC-61
Test case Name:	Submit Extension Request
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how a student submits a request for extension of research deadline
Primary Actor:	<Student>

Main Success Scenario:

#	User Action	#	System Response
1.	Student opens Extension Request page		
		2.	System displays extension request form with required fields
3.	Student enters extension reason and requested duration		
		4.	System validates duration format and checks against policy limits
5.	Student uploads supporting document		
		6.	System validates file format and size
7.	Student clicks “Submit Request”		
		8.	System saves request in database with status “Submitted”
		9.	System notifies supervisor and admin via email and system notification.
		10.	System displays “Extension Request Submitted Successfully”

Testing Requirement:

Testing Condition:	Student is logged in; Student has an active research timeline
Input Data:	enters extension reason and requested duration; uploads supporting document; clicks “Submit Request”
Expected Result:	Extension request is submitted for review

Actual Result:	Extension request is submitted for review
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-62	
Test case Name:	Review Extension Request	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a supervisor reviews and decides on extension requests	
Primary Actor:	<Supervisor>	

Main Success Scenario:

#	User Action	#	System Response
1.	Supervisor opens extension requests		
		2.	System displays pending requests
3.	Supervisor selects a request		
		4.	System displays request details

5.	Supervisor reviews and adds comments		
6.	Supervisor selects decision		
7.	Supervisor submits decision		
		8.	System updates status and notifies users

Testing Requirement:

Testing Condition:	Supervisor is logged in
Input Data:	selects a request; selects decision
Expected Result:	Extension request is resolved and parties are notified
Actual Result:	Extension request is resolved and parties are notified
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-63	
Test case Name:	Approve Extension Request	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025

Test case Description:	This use case describes how the admin approves a student's extension request in the system
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects extension request		
		2.	System displays request details
3.	Admin selects "Approve"		
		4.	System updates request status
		5.	System saves approval record
		6.	System notifies student and supervisor

Testing Requirement:

Testing Condition:	Extension request must be reviewed; Admin must be logged in
Input Data:	selects extension request; selects "Approve"
Expected Result:	Extension request status is updated to Approved
Actual Result:	Extension request status is updated to Approved
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-64
Test case Name:	Reject Extension Request
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin rejects a student's extension request with proper justification
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects extension request		
		2.	System displays request details
3.	Admin selects "Reject"		
4.	Admin enters rejection remarks		
		5.	System saves rejection decision
		6.	System updates request status
		7.	System notifies student and supervisor

Testing Requirement:

Testing Condition:	Extension request must be reviewed; Admin must be logged in
Input Data:	selects extension request; selects “Reject”; enters rejection remarks
Expected Result:	Extension request status is updated to Rejected
Actual Result:	Extension request status is updated to Rejected
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-65
Test case Name:	Extension Case Reporting
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin generates reports related to extension cases
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects extension reporting		

		2.	System displays report options
3.	Admin selects criteria		
		4.	System generates report
		5.	System displays report

Testing Requirement:

Testing Condition:	Extension case records must exist
Input Data:	selects extension reporting; selects criteria
Expected Result:	Extension case report is generated successfully
Actual Result:	Extension case report is generated successfully
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-66
Test case Name:	Extension Case Record Management
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin manages extension case records

Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin accesses extension records		
		2.	System displays extension case list
3.	Admin manages extension records		
		4.	System saves updates

Testing Requirement:

Testing Condition:	Admin must be logged in
Input Data:	N/A
Expected Result:	Extension case records are maintained successfully
Actual Result:	Extension case records are maintained successfully
Priority:	Medium
Frequency:	Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-67
Test case Name:	Submit Degree Issuance Request

Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how a student requests degree issuance after completing all requirements
Primary Actor:	<Student>

Main Success Scenario:

#	User Action	#	System Response
1.	Student selects “Degree Issuance Request”		
		2.	System displays request form
3.	Student submits request		
		4.	System validates request
		5.	System saves degree request
		6.	System notifies admin

Testing Requirement:

Testing Condition:	Student is logged in; Student has completed all research requirements (thesis, viva, etc.)
Input Data:	selects “Degree Issuance Request”
Expected Result:	Degree issuance request is saved with Pending Verification status
Actual Result:	Degree issuance request is saved with Pending Verification status

Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-68
Test case Name:	Verify Degree Requirements
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how admin verifies student's eligibility for degree issuance
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects degree request		
		2.	System displays academic records
3.	Admin verifies requirements		
		4.	System updates verification status

Testing Requirement:

Testing Condition:	Admin is logged in; Degree request is submitted by student
Input Data:	selects degree request
Expected Result:	Degree requirements are verified successfully
Actual Result:	Degree requirements are verified successfully
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-69
Test case Name:	Generate Degree Letter File
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case explains how the system generates a degree letter after verification
Primary Actor:	<System><Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects “Generate Degree Letter”		

		2.	System generates degree letter file
		3.	System stores degree letter

Testing Requirement:

Testing Condition:	Degree request is verified; Admin is logged in
Input Data:	selects “Generate Degree Letter”
Expected Result:	Degree letter file is generated successfully
Actual Result:	Degree letter file is generated successfully
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-70
Test case Name:	Approve Degree Letter
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin approves the generated degree letter
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin reviews degree letter		
2.	Admin selects “Approve”		
		3.	System updates approval status
		4.	System notifies student

Testing Requirement:

Testing Condition:	Degree letter must be generated
Input Data:	selects “Approve”
Expected Result:	Degree letter is approved and finalized
Actual Result:	Degree letter is approved and finalized
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-71
Test case Name:	Reject Degree Letter
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025

Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin rejects a degree letter due to issues or missing information
Primary Actor:	<Admin>

Main Success Scenario:

#	User Action	#	System Response
1.	Admin reviews degree letter		
2.	Admin selects “Reject”		
		3.	System updates rejection status
		4.	System notifies student

Testing Requirement:

Testing Condition:	Degree letter must be generated
Input Data:	selects “Reject”
Expected Result:	Degree letter is rejected with remarks
Actual Result:	Degree letter is rejected with remarks
Priority:	High
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-72
Test case Name:	Download Degree Letter
Test case Prepared by:	Sana Tariq Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how a student downloads the approved degree letter
Primary Actor:	<Student>

Main Success Scenario:

#	User Action	#	System Response
1.	Student selects “Download Degree Letter”		
		2.	System retrieves degree letter
		3.	System downloads file

Testing Requirement:

Testing Condition:	Degree letter must be approved
Input Data:	selects “Download Degree Letter”
Expected Result:	Degree letter is downloaded successfully
Actual Result:	Degree letter is downloaded successfully

Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Test case ID:	TC-73	
Test case Name:	Degree Letter Reporting	
Test case Prepared by:	Sana Tariq	Test case Prepared on: 29-Dec-2025
Test case updated by:	Alishba Arshad	Test case updated on: 29-Dec-2025
Test case Description:	This use case describes how the admin generates reports related to degree letters	
Primary Actor:	<Admin>	

Main Success Scenario:

#	User Action	#	System Response
1.	Admin selects degree reporting		
		2.	System displays report options
3.	Admin selects criteria		
		4.	System generates report
		5.	System displays report

Testing Requirement:

Testing Condition:	Degree letter records must exist
Input Data:	selects degree reporting; selects criteria
Expected Result:	Degree letter is generated successfully
Actual Result:	Degree letter is generated successfully
Priority:	Medium
Frequency:	Less Frequently Use
Test Acceptance:	Passed

Chapter#6

User Manual

6.1 Screenshots:

6.1.1 Home Page

The Home Page is the first screen users see when they visit the GRMS website. It provides an overview of the system and allows users to navigate to different sections.

Features:

- Navigation bar with links to Home, About, Features, Help, and Login
- Call-to-action buttons: "Get Started" and "Learn More"
- Clean and professional design

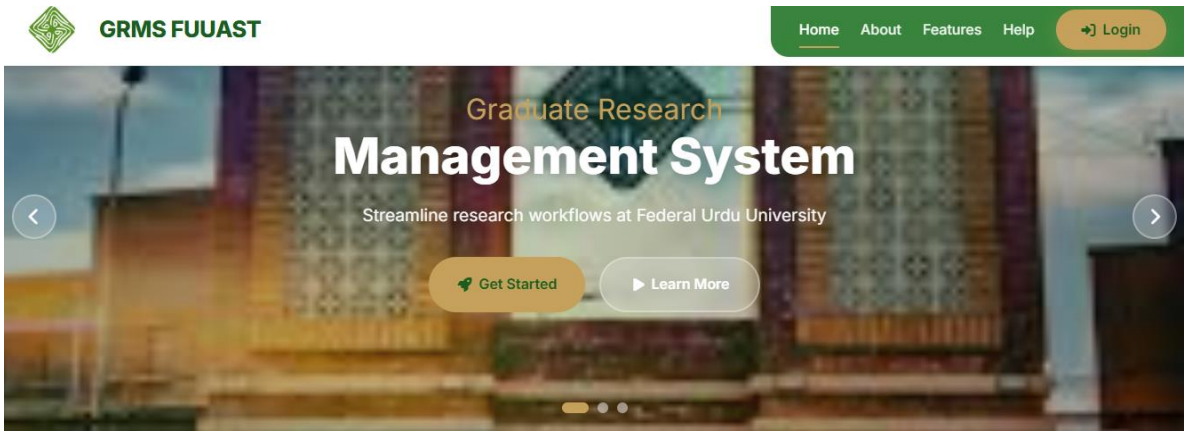

Figure 6.1: GRMS Home Page

6.1.2 Login Page

The Login Page allows registered users to access their accounts. Users must enter their username and password to sign in.

Features:

- Username and password fields
- "Sign In" button
- "Create Account" link for new users
- "Forgot Password" link for password recovery

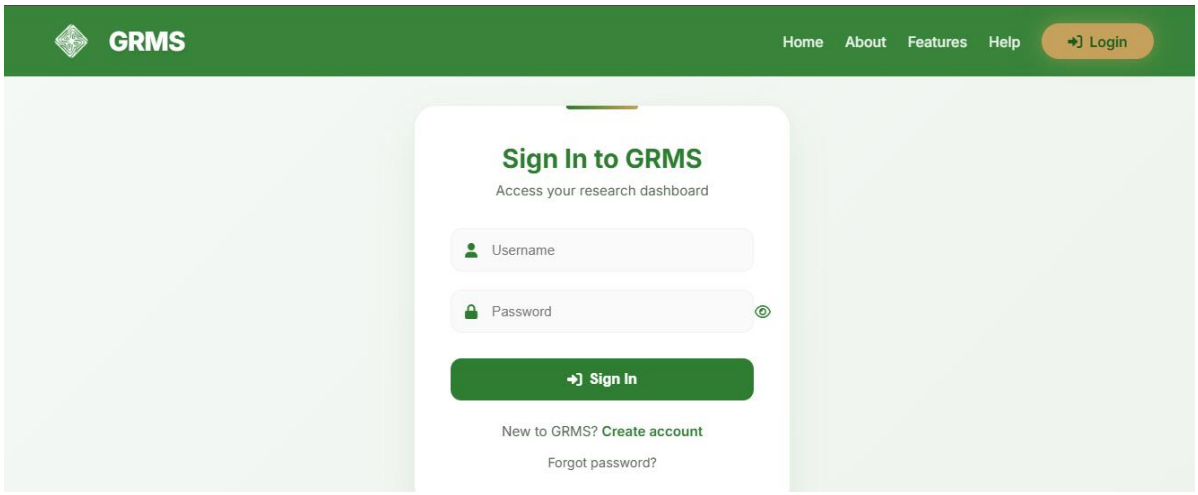

Figure 6.2: GRMS Login Page

6.1.3 Registration Page

The Registration Page allows new users to create an account. Users must fill in all required fields to register.

Features:

- Full Name field
- Username field
- Email Address field
- Role selection (dropdown)
- Password with validation requirements (minimum 8 characters, 1 uppercase, 1 lowercase, 1 number)
- Confirm Password field
- "Create Account" button
- Terms and Privacy Policy agreement

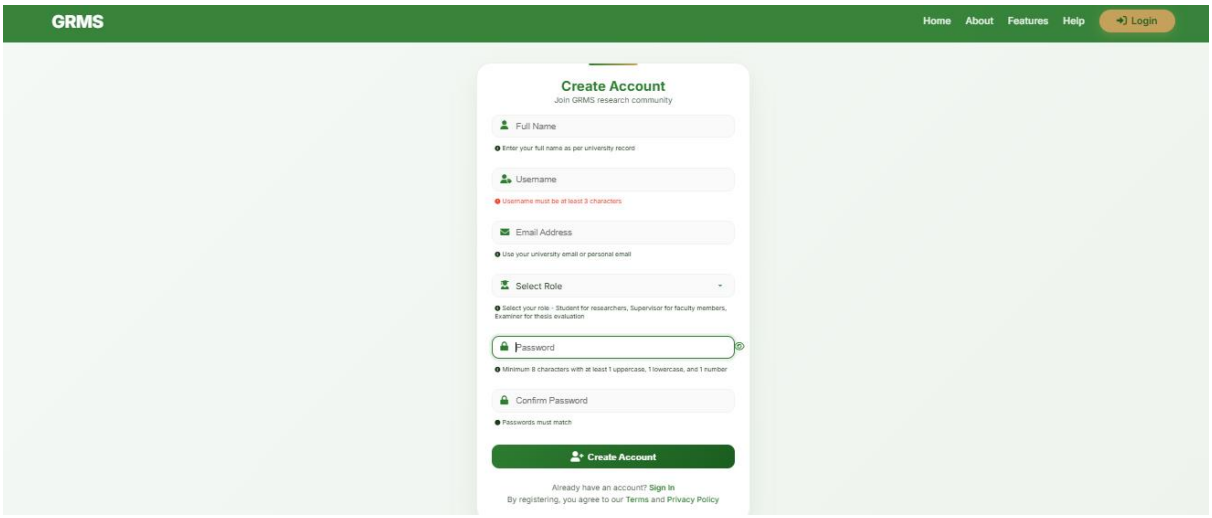

Figure 6.3: GRMS Registration Page

6.1.4 Admin Dashboard

The Admin Dashboard provides a comprehensive overview of the system. It displays key statistics and allows administrators to manage all aspects of the research management system.

Features:

- Total Students count with Active status
- Supervisors count with Available status
- Extension Requests with Urgent count
- Degree Requests with Pending verification status
- Notifications section
- Quick Actions buttons:
    - Add Student
    - Add Supervisor
    - Pending Synopsis
    - Extensions
- Filter Students by Department
- Students List with details:

- Registration No
- Name
- Department
- Program
- Session
- Supervisor
- Status

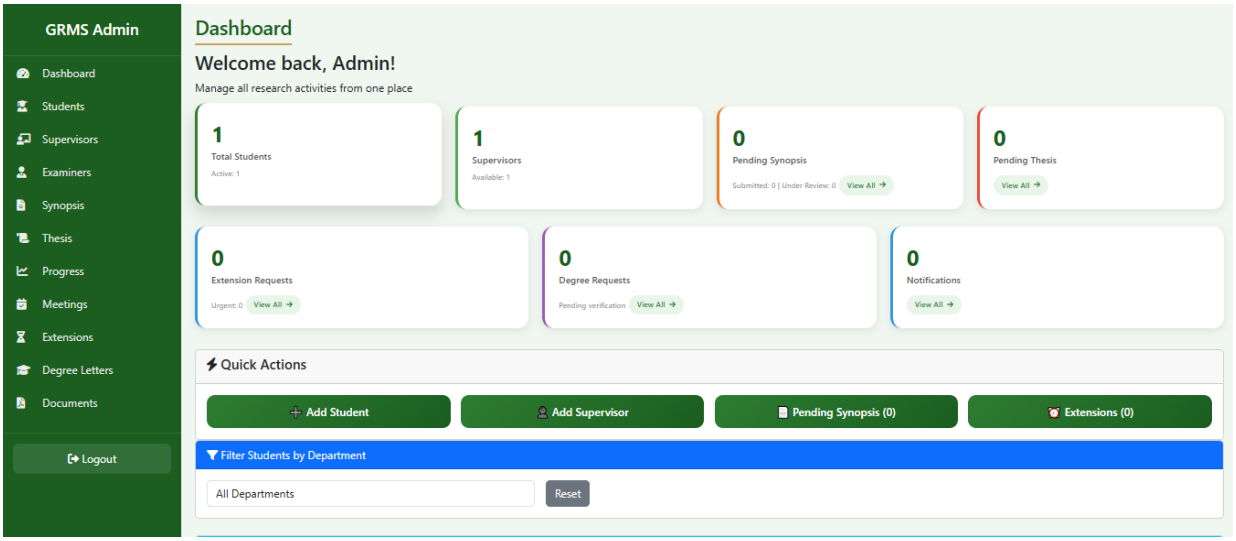

Figure 6.4: GRMS Admin Dashboard

6.1.5 Student Dashboard

The Student Dashboard provides students with a personalized view of their research progress and academic journey.

Features:

- Welcome message with student name
- Profile Management section
    - Student Name
    - Registration Number
    - "View Complete Profile" button

- Sidebar Navigation:
    - Dashboard
    - My Synopsis
    - Progress Reports
    - Thesis
    - Meetings
    - Extensions
    - Degree Letter
    - Logout
- Quick Stats:
    - Progress Reports
    - Pending Reports
    - Meetings
    - Extensions

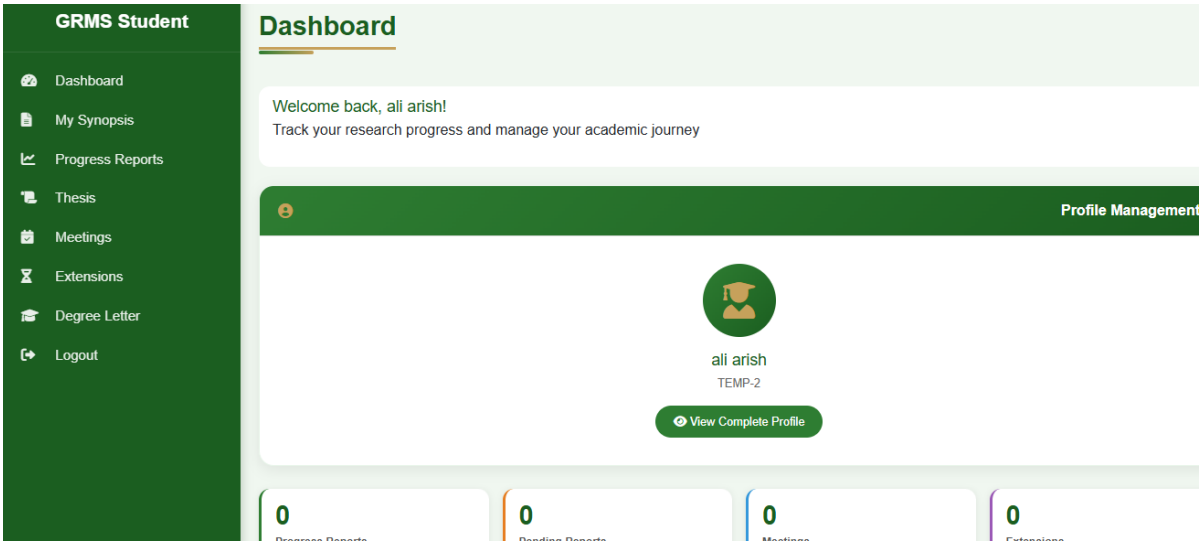

Figure 6.5: GRMS Student Dashboard

6.2 User Manual

6.2.1 User Roles

GRMS supports three main user roles, each with specific permissions and access levels.

1. Student

Register and login

Submit synopsis

Submit progress reports

Submit thesis

Schedule meetings

Request extensions

Request degree letter

2. Supervisor

Review synopsis

Review progress reports

Review thesis

Schedule meetings with students

Verify degree letters

3. Admin

Manage all users (students, supervisors, examiners)

Manage products and documents

View and manage orders

Manage extensions

Manage degree letters

Full system control

6.2.2 Registration Process

For Students:

1. Click "Create Account"
2. Enter Full Name, Username, Email, Role (Student)

3. Create Password (8+ characters, uppercase, lowercase, number)
4. Confirm Password
5. Agree to Terms & Privacy Policy
6. Click "Create Account"

6.2.3 Login Process

1. Click "Login" in navigation bar
2. Enter Username and Password
3. Click "Sign In"
4. Redirected to respective dashboard

Forgot Password:

1. Click "Forgot password?" on login page
2. Enter registered email
3. Follow instructions sent to email
4. Reset password

6.2.4 Admin Dashboard Guide

Quick Actions:

- Add Student
- Add Supervisor
- Pending Synopsis
- Extensions

Managing Students:

1. Go to Students from sidebar
2. View complete student list
3. Filter by Department
4. Click student to view/update details
5. Use "Add Student" to register new student

6.2.5 Student Dashboard Guide

Profile Management:

1. Click "View Complete Profile"
2. Update personal information
3. Save changes

My Synopsis:

1. Navigate to My Synopsis
2. Click "Submit New Synopsis"
3. Fill details and upload PDF
4. Track status

Thesis:

1. Navigate to Thesis
2. Click "Submit Thesis"
3. Fill details and upload PDF
4. Track status

Progress Reports:

1. Navigate to Progress Reports
2. Click "Submit Progress Report"
3. Select semester and upload PDF
4. View supervisor feedback

Meetings:

1. Navigate to Meetings
2. View scheduled meetings
3. Click "Schedule Meeting"
4. Enter details and agenda

Extensions:

1. Navigate to Extensions
2. Click "Request Extension"
3. Provide reason and duration
4. Upload supporting document
5. Track status

Degree Letter:

1. Navigate to Degree Letter
2. Click "Request Degree Letter"
3. Wait for verification

6.2.6 Document Upload (Admin)

1. Navigate to Documents → Upload New Document
2. Select student
3. Select document type (Admission/Enrollment Letter)
4. Enter title
5. Upload PDF (Max 5MB)
6. Set expiry date (optional)
7. Click "Upload Document"

6.2.7 Logout

1. Click username/profile icon
2. Select "Logout"
3. Redirected to login page